

Fake News Detection

Group : Group 2B

Contributors :

1. Daniel Mwaka
2. Abdirahman Yussuf
3. Eliana Kariuki
4. Gerald Wanjala
5. Marion Osong

1. Introduction

The digital age is characterized by the rapid and widespread dissemination of information. The dissemination of news articles with implausible, dubious, dishonest or unreliable content propagates misinformation, erodes public trust, and manipulates public opinion. Manual fact-checking is unfeasible and unviable owing to the sheer volume of "informational textual content" posted across news dissemination websites and shared across social media platforms. This project presents a robust machine learning framework designed to automatically detect fake news by classifying news articles as either genuine (Class 1) or fabricated (Class 0). The methodology centers on natural language processing (NLP), Logistic Regression, Ensemble models and Deep Learning to analyze the linguistic features, sentiment, and contextual features in text data.

1.1 Business Understanding

In today's information-driven society, the spread of misinformation and fake news through online news dissemination platforms and social media platforms has become a serious threat to public trust, democracy, health communication, and media integrity. The ability to automatically distinguish fake from real news is essential for:

- Social media platforms (e.g., Meta, Twitter/X, Reddit).
- Fact-checking organizations.
- News aggregators (e.g., Google News, Apple News).
- The general public.

1.2 Problem Statement:

The proliferation of fake news on digital platforms poses a significant risk to public trust and decision-making. Traditional methods of identifying fake news are slow and resource-intensive, making it difficult to manage the vast volume of online content.

1.3 Proposed solution

This project aims to develop an automated binary classification system using transformer-based Natural Language Processing (NLP) models to detect and flag fake news articles with high accuracy. By leveraging advanced NLP techniques, the system will offer a

scalable solution to help identify harmful misinformation, improving information credibility and supporting the fight against digital manipulation.

1.4 Objectives

1. Perform Exploratory Data Analysis.
2. Build a Baseline Logistic Regression Model to predict class of an article.
3. Build an XGBoost model, a LSTM neural network and finetune the RoBERTa-base Transformer.
4. Interpret best performing ML model using the LIME library.
5. Deploy selected model using FastAPI and Streamlit.

1.5 Success Criteria

The **Recall (Fake)** metric is set as the success criteria because the project aims to **catch as many fake articles as possible**. We aim to achieve a recall score (**Recall (Fake)**) and

2. Data Loading, Feature Engineering, and Preprocessing

2.1 Data Loading

```
In [1]: ▶ # Declare dataset path
import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

/kaggle/input/fake-news-detection/data.csv
```

```
In [2]: ▶ # Declare images directory
import os
images_dir = '/kaggle/working/images'
os.makedirs(images_dir, exist_ok=True)
print(os.listdir('/kaggle/working/'))

['.virtual_documents', 'images']
```

```
In [3]: ▶ # !pip install -U scikit-learn==1.5.2 imbalanced-learn==0.12.3 xgboost==1.4.2
```



```
In [4]: # Import basic libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from wordcloud import WordCloud
from collections import Counter
import io

# Import NLP tools and NLTK library modules
import nltk
from nltk.corpus import stopwords, wordnet
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.stem import WordNetLemmatizer, PorterStemmer
from nltk import pos_tag
from nltk.util import bigrams
from nltk.util import trigrams
import string
import re
import spacy

# Import scikit-learn library's classes, tools, and modules
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.metrics import accuracy_score, classification_report
from sklearn.metrics import f1_score, precision_score, recall_score
from sklearn.linear_model import LogisticRegression
from xgboost import XGBClassifier
from sklearn.utils.class_weight import compute_class_weight
from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import RandomUnderSampler
import joblib
from gensim.models import Word2Vec

# Import tensorflow, and keras modules
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Model, load_model
from tensorflow.keras.layers import TextVectorization, Embedding,
from tensorflow.keras.layers import Dropout, Dense, LSTM, Input,
from tensorflow.keras.metrics import Precision, Recall, AUC
from transformers import TFRobertaForSequenceClassification, RobertaForSequenceClassification
from transformers import logging as transformers_logging

# Import transformers and pytorch modules
from transformers import RobertaTokenizerFast, RobertaForSequenceClassification
from datasets import Dataset
import torch
from torch.utils.data import Dataset
import os

# Import Model Interpretation Libraries and modules
from lime.lime_text import LimeTextExplainer
import matplotlib.patches as mpatches
```

```
# Disable warnings
import warnings
warnings.filterwarnings('ignore')
```

```
2025-08-18 17:12:14.386477: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:477] Unable to register cuFFT factory: Attempting to register factory for plugin cuFFT when one has already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
E0000 00:00:1755537134.611201          97 cuda_dnn.cc:8310] Unable to register cuDNN factory: Attempting to register factory for plugin cuDNN when one has already been registered
E0000 00:00:1755537134.677933          97 cuda_blas.cc:1418] Unable to register cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has already been registered
```

```
In [5]: data = pd.read_csv("/kaggle/input/fake-news-detection/data.csv")
data.head()
```

```
Out[5]:
```

	title	extracted_article_text	news_type	class
0	comments at the CNN Republican presidential de...	CQ Username Password Stay signed in Forgot you...	political	1
1	Hollywood Club Crawl	Submit all required information below and rece...	gossip	1
2	FORMER CIA AGENT SAYS OBAMA WORKING WITH MUSLI...	Former CIA Agent, Clare Lopez has been ringing...	political	0
3	Flashback to Golden Globe Nominee Emma Stone's...	Introducing Monoclean: Focus on Your Life, Not...	gossip	1
4	BREAKING: DONALD TRUMP Video Statement On Leak...		political	0

```
In [6]: # Check shape
data.shape
```

```
Out[6]: (24455, 4)
```

```
In [7]: # Check duplicates
data.duplicated().sum()
```

```
Out[7]: 1990
```

```
In [8]: # Drop duplicates
data.drop_duplicates(inplace=True)
```

```
In [9]: # Confirm no duplicates
data.duplicated().sum()
```

```
Out[9]: 0
```

```
In [10]: ▶ # Inspect column attributes to check missing values
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 22465 entries, 0 to 24454
Data columns (total 4 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   title                                22465 non-null   object
1   extracted_article_text               22465 non-null   object
2   news_type                           22465 non-null   object
3   class                               22465 non-null   int64
dtypes: int64(1), object(3)
memory usage: 877.5+ KB
```

- Data was successfully loaded and duplicates removed.
- There are no entries with missing values

2.2 Feature Engineering

```
In [12]: ▶ # Engineer the article feature
data['article'] = data['title'] + " " + data['extracted_article_t

# Engineer the text length
data['num_sentences']=data['extracted_article_text'].apply(lambda

# Engineer the text length
data['text_length']=data['extracted_article_text'].apply(lambda x
data.head(2)
```

Out[12]:

	title	extracted_article_text	news_type	class	article	num_sentences	text_length
--	-------	------------------------	-----------	-------	---------	---------------	-------------

0	comments at the CNN Republican presidential de...	CQ Username Password Stay signed in Forgot you...	political	1	comments at the CNN Republican presidential de...	3	
1	Hollywood Club Crawl	Submit all required information below and rece...	gossip	1	Hollywood Club Crawl Submit all required infor...	58	

2.3 Data Preprocessing

- Define a function for **text cleaning, normalization, word tokenizing, Part-of-Speech tagging, lemmatization** and **removing stopwords**, using the **spaCy** library.

```
In [13]: ▶ # Load spaCy model
nlp = spacy.load('en_core_web_sm', disable=['parser', 'ner'])
# Define function to clean text using spaCy
def cleaned_text(text, debug=False):
    # Convert lists to strings if necessary
    if isinstance(text, list):
        text = " ".join(text)
    # Ensure text is a string
    if not isinstance(text, str):
        text = str(text)
    text = text.lower() # Lowercase all characters
    text = re.sub(r'https?://\S+|www\.\S+', '', text) # Remove URLs
    text = re.sub(r'\.[*?\\]', '', text) # Remove content enclosed in brackets
    text = re.sub(r'<.*?>+', '', text) # Remove content enclosed in HTML tags
    text = re.sub(r'\n', ' ', text) # Replace newline characters with spaces

    # Define chunk size
    chunk_size = 900000
    cleaned_tokens = []
    # Process text in chunks if it's too long
    if len(text) > chunk_size:
        chunks = [text[i:i + chunk_size] for i in range(0, len(text), chunk_size)]
    else:
        chunks = [text]

    for chunk in chunks:
        doc = nlp(chunk)
        # Extract lemmatized tokens, excluding stopwords, and punctuation
        for token in doc:
            if (not token.is_stop and
                not token.is_punct and
                not token.is_digit and
                len(token.lemma_) > 2):
                # Remove any remaining non-alphabetic characters
                cleaned = re.sub(r'[^a-zA-Z]', '', token.lemma_)
                if cleaned: # Only include non-empty strings
                    cleaned_tokens.append(cleaned)

    return cleaned_tokens

# Apply the clean_text function to the 'spacy' column
data['spacy'] = data['article'].apply(cleaned_text)
```

```
In [14]: ▶ # Join tokens
data['text'] = data['spacy'].apply(lambda x: ' '.join(x))
data.head(2)
```

```
Out[14]:
```

	title	extracted_article_text	news_type	class	article	num_sentences	text
0	comments at the CNN Republican presidential de...	CQ Username Password Stay signed in Forgot you...	political	1	comments at the CNN Republican presidential de...	3	
1	Hollywood Club Crawl	Submit all required information below and rece...	gossip	1	Hollywood Club Crawl Submit all required infor...	58	

```
In [19]: ▶ # Preview a random sample
data.iloc[2555, 8]
```

```
Out[19]: 'democrat congresswoman obama work hand hand islamic extremist s
end troop harm way s pay attention s happen middle east s talk r
eckless action dangerous man world bill maher tonight speak demo
cratic congresswoman tulsi gabbard issue agree frustrating refus
al president obama accept islamic extremism global threatgabbard
say s crazy s important identify enemy defeat say s ridiculous b
lame poverty thrill seek give house win t solve problemmaher bri
ng sensitive liberal mere suggestion certain islamic belief cons
ider barbaric western world gabbard agree kind liberal philosoph
y get way able address matter maher tonight speak democratic con
gresswoman tulsi gabbard issue agree frustrating refusal preside
nt obama accept islamic extremism global threatgabbard say s cra
zy s important identify enemy defeat say s ridiculous blame pove
rty thrill seek give house win t solve problemmaher bring sensit
ive liberal mere suggestion certain islamic belief consider barb
aric western world gabbard agree kind liberal philosophy get way
able address mattersbill maher go ask dem rep obama win t islami
c extremism gabbard recall treat woman go iraq soldier us try tr
ain happy ending mediaite'
```

```
In [20]: ▶ data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 22465 entries, 0 to 24454
Data columns (total 9 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   title                                22465 non-null  object
1   extracted_article_text               22465 non-null  object
2   news_type                           22465 non-null  object
3   class                               22465 non-null  int64
4   article                             22465 non-null  object
5   num_sentences                       22465 non-null  int64
6   text_length                         22465 non-null  int64
7   spacy                               22465 non-null  object
8   text                                22465 non-null  object
dtypes: int64(3), object(6)
memory usage: 1.7+ MB
```

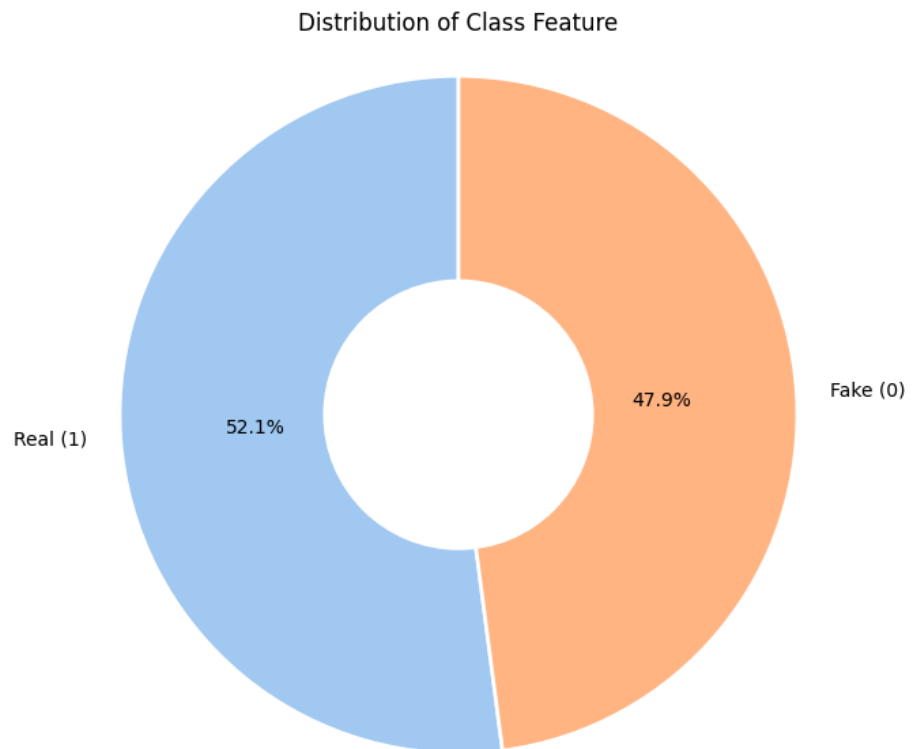
Objective 1: Perform Extraplorary Data Analysis

Target Variable Class Distributions

```
In [21]: ▶ # Print value counts for each class
data['class'].value_counts()
```

```
Out[21]: class
1      11699
0      10766
Name: count, dtype: int64
```

```
In [69]: ▶ # Visualize Class Distribution
class_counts = data['class'].value_counts()
plt.figure(figsize=(8, 6))
# Plot pie chart with a wedgeprops to create the donut effect
plt.pie(class_counts, labels=['Real (1)', 'Fake (0)'], autopct='%
        colors=sns.color_palette('pastel'), wedgeprops={'edgecolor
centre_circle = plt.Circle((0, 0), 0.4, fc='white')
fig = plt.gcf()
fig.gca().add_artist(centre_circle)
plt.axis('equal')
plt.title('Distribution of Class Feature')
plt.tight_layout()
plt.savefig('/kaggle/working/images/class-distributions.png', dp
plt.show()
```

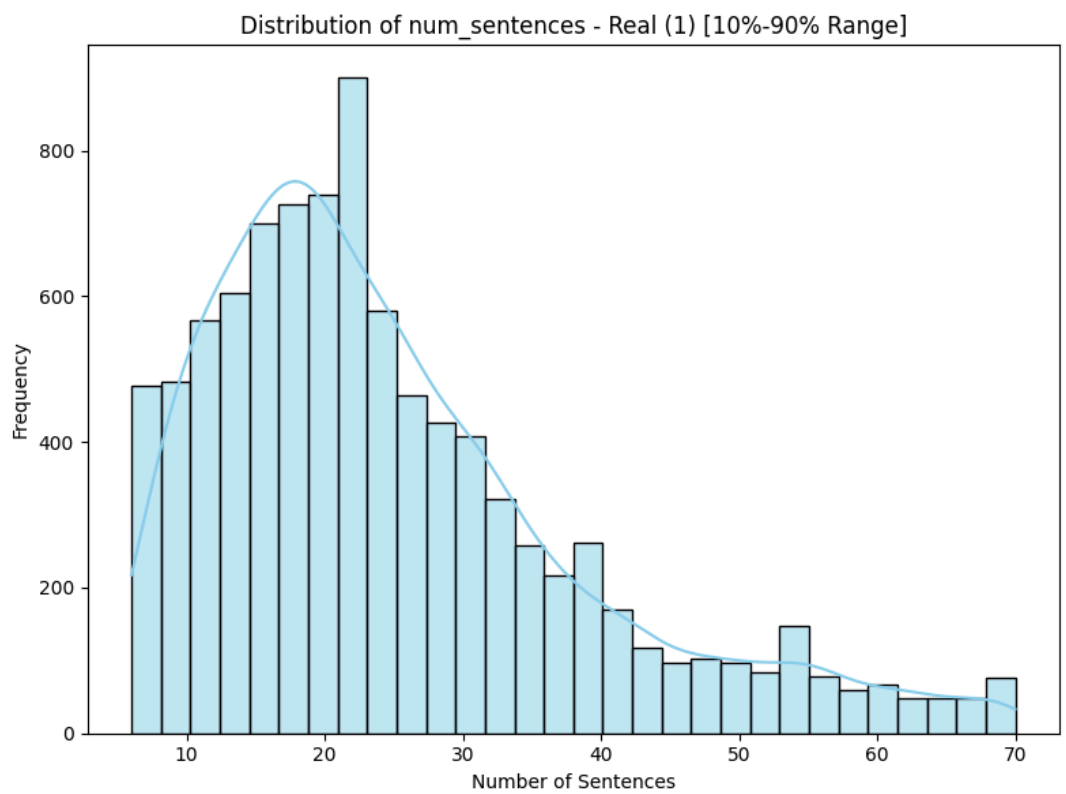
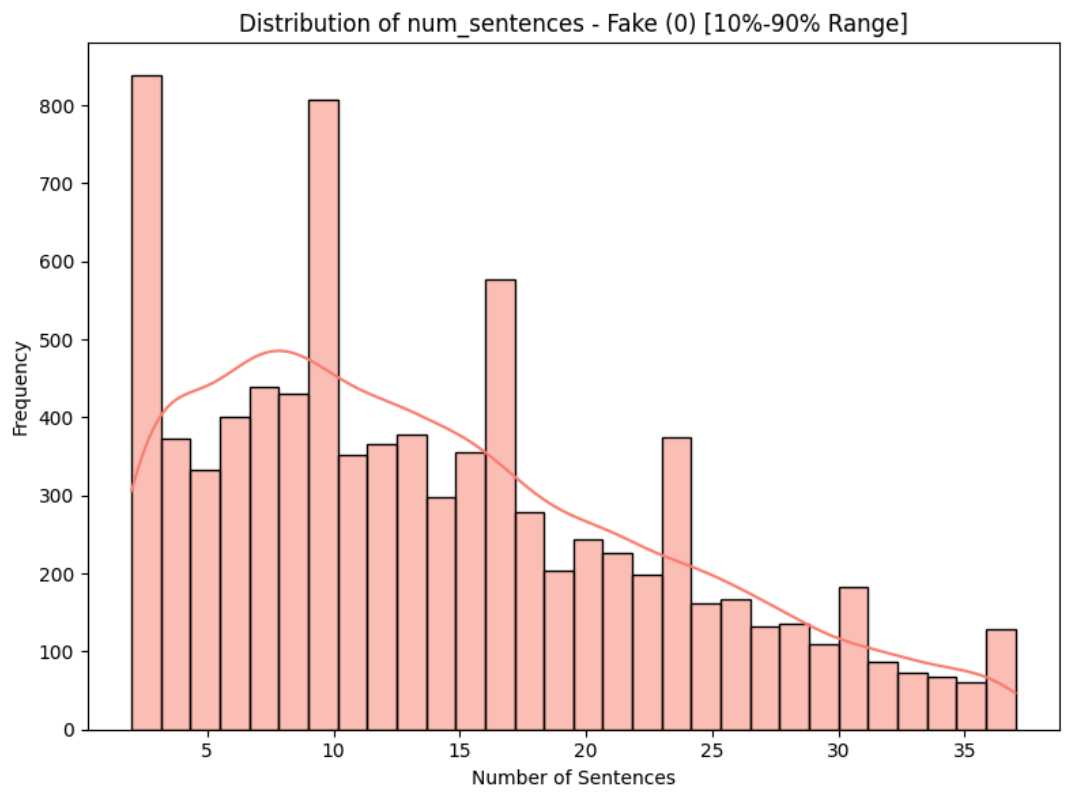


num_sentences Distribution

```
In [66]: ► # Filter between 10% and 90% percentiles to avoid skewing by outliers
def filter_percentile(df, col):
    lower = df[col].quantile(0.10)
    upper = df[col].quantile(0.90)
    return df[(df[col] >= lower) & (df[col] <= upper)]

# Plot distribution for class 0 (filtered data)
class0_data = filter_percentile(data[data['class'] == 0], 'num_sentences')
plt.figure(figsize=(8, 6))
sns.histplot(
    data=class0_data,
    x='num_sentences',
    kde=True,
    bins=30,
    color='salmon')
plt.title('Distribution of num_sentences - Fake (0) [10%-90% Range Filtered]')
plt.xlabel('Number of Sentences')
plt.ylabel('Frequency')
plt.tight_layout()
plt.savefig('./images/num_sentences_distribution_class0_filtered.png')
plt.show()

# Plot distribution for class 1 (filtered data)
class1_data = filter_percentile(data[data['class'] == 1], 'num_sentences')
plt.figure(figsize=(8, 6))
sns.histplot(
    data=class1_data,
    x='num_sentences',
    kde=True,
    bins=30,
    color='skyblue')
plt.title('Distribution of num_sentences - Real (1) [10%-90% Range Filtered]')
plt.xlabel('Number of Sentences')
plt.ylabel('Frequency')
plt.tight_layout()
plt.savefig('./images/num_sentences_distribution_class1_filtered.png')
plt.show()
```

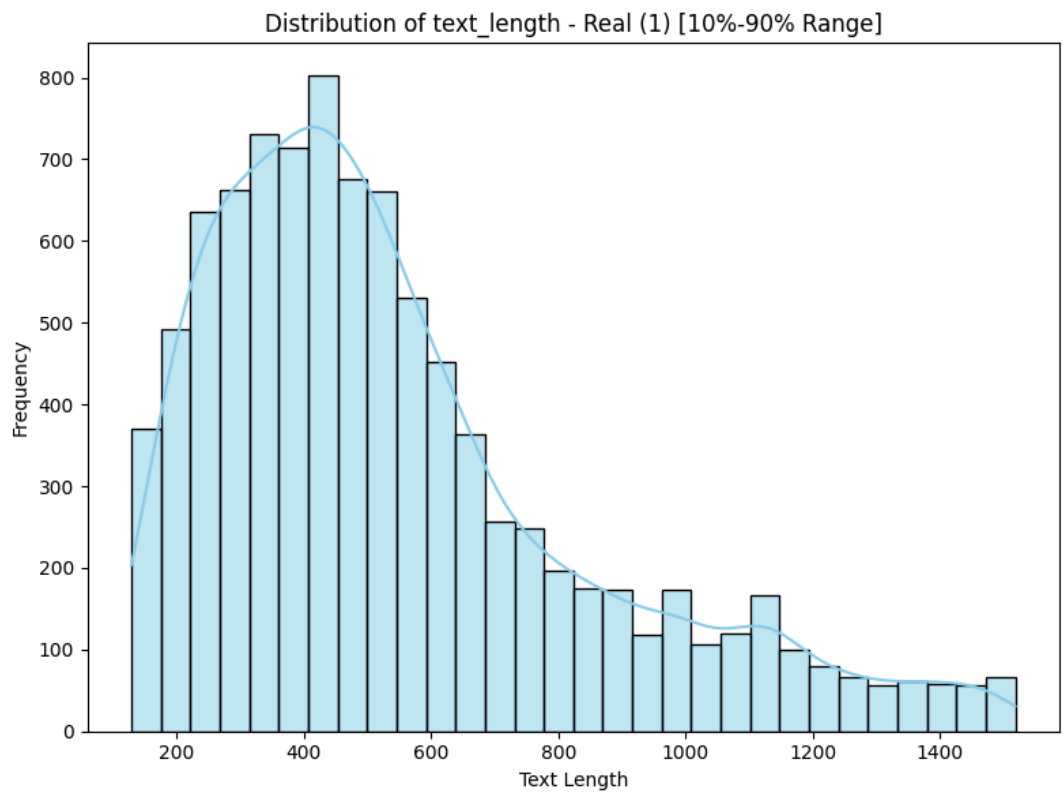
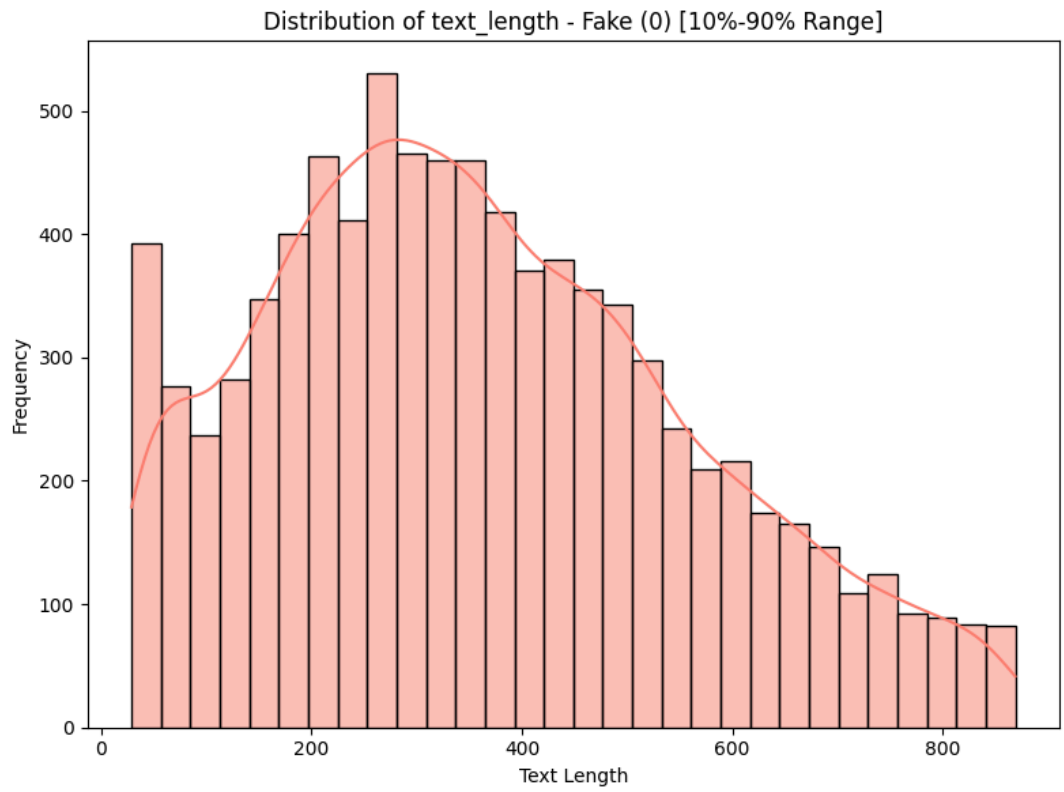


text_length Distribution

```
In [67]: ▶ # Filter between 10% and 90% percentiles to avoid skewing by outliers
def filter_percentile(df, col):
    lower = df[col].quantile(0.10)
    upper = df[col].quantile(0.90)
    return df[(df[col] >= lower) & (df[col] <= upper)]

# Plot distribution for class 0 (filtered data)
class0_data = filter_percentile(data[data['class'] == 0], 'text_length')
plt.figure(figsize=(8, 6))
sns.histplot(
    data=class0_data,
    x='text_length',
    kde=True,
    bins=30,
    color='salmon')
plt.title('Distribution of text_length - Fake (0) [10%-90% Range]')
plt.xlabel('Text Length')
plt.ylabel('Frequency')
plt.tight_layout()
plt.savefig('./images/text_length_distribution_class0_filtered.png')
plt.show()

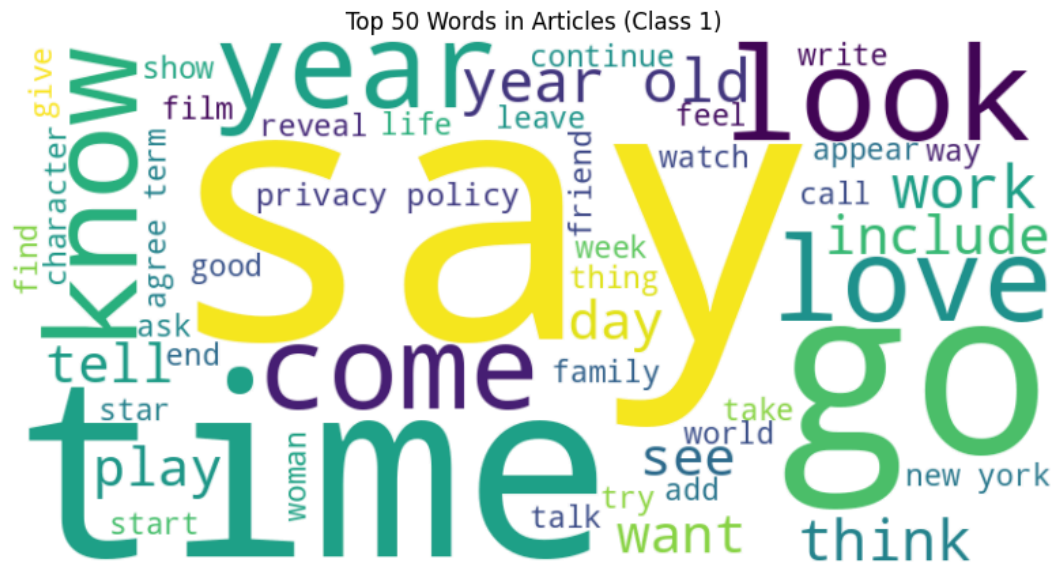
# Plot distribution for class 1 (filtered data)
class1_data = filter_percentile(data[data['class'] == 1], 'text_length')
plt.figure(figsize=(8, 6))
sns.histplot(
    data=class1_data,
    x='text_length',
    kde=True,
    bins=30,
    color='skyblue')
plt.title('Distribution of text_length - Real (1) [10%-90% Range]')
plt.xlabel('Text Length')
plt.ylabel('Frequency')
plt.tight_layout()
plt.savefig('./images/text_length_distribution_class1_filtered.png')
plt.show()
```



Most Frequent/ Significant Words Among Fake News Articles (Label 0)


```
In [27]: ▶ # Filter data for class 1
class_1_data = data[data['class'] == 1]

# Combine all text from extracted_article_text
text = ' '.join(class_1_data['text'].dropna().astype(str))
wordcloud = WordCloud(width=800, height=400, background_color='white')
plt.figure(figsize=(10, 5))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Top 50 Words in Articles (Class 1)')
plt.savefig('/kaggle/working/images/top-50-words-for-class1-wordcloud.png')
plt.show()
```



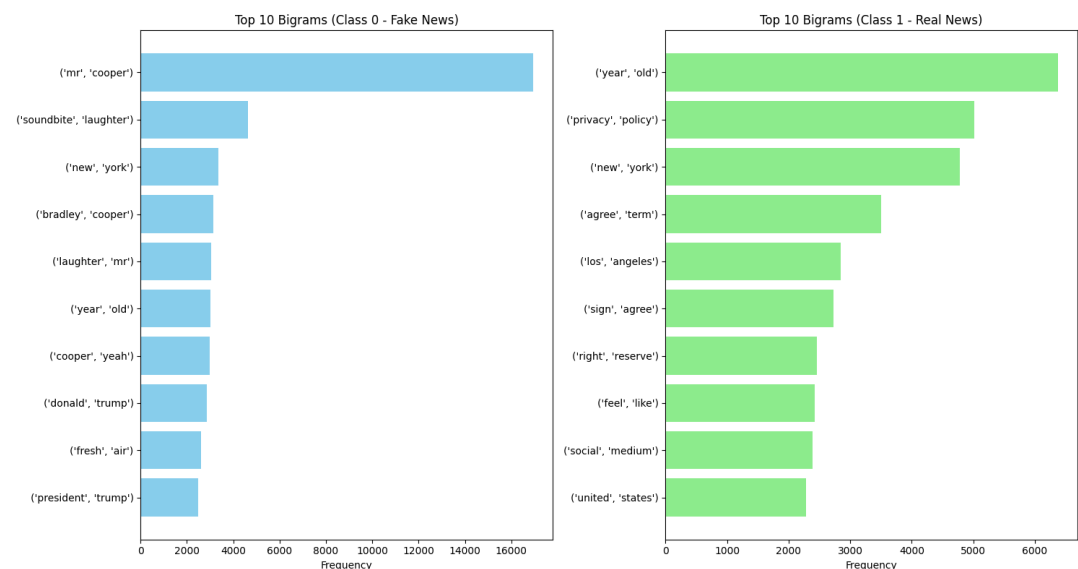
Top-10 Bigrams

```
In [25]: ▶ # Filter and plot top-10 bigrams for both fake and real news articles
def get_top_bigrams(text_series):
    all_bigrams = []
    for text in text_series.dropna():
        tokens = word_tokenize(text.lower())
        bigram_list = list(bigrams(tokens))
        all_bigrams.extend(bigram_list)
    return Counter(all_bigrams).most_common(10)

class_0_data = data[data['class'] == 0]['text']
class_1_data = data[data['class'] == 1]['text']

# Get top 10 bigrams for each class
top_bigrams_class_0 = get_top_bigrams(class_0_data)
top_bigrams_class_1 = get_top_bigrams(class_1_data)
bigrams_0, counts_0 = zip(*sorted(top_bigrams_class_0, key=lambda x: x[1]))
bigrams_1, counts_1 = zip(*sorted(top_bigrams_class_1, key=lambda x: x[1]))

# Create subplots
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 8))
# Plot for class 0
ax1.barh([str(bg) for bg in bigrams_0], counts_0, color='skyblue')
ax1.set_title('Top 10 Bigrams (Class 0 - Fake News)')
ax1.set_xlabel('Frequency')
# Plot for class 1
ax2.barh([str(bg) for bg in bigrams_1], counts_1, color='lightgreen')
ax2.set_title('Top 10 Bigrams (Class 1 - Real News)')
ax2.set_xlabel('Frequency')
plt.tight_layout()
plt.savefig('/kaggle/working/images/top-10-bigrams.png', dpi=300,
plt.show())
```

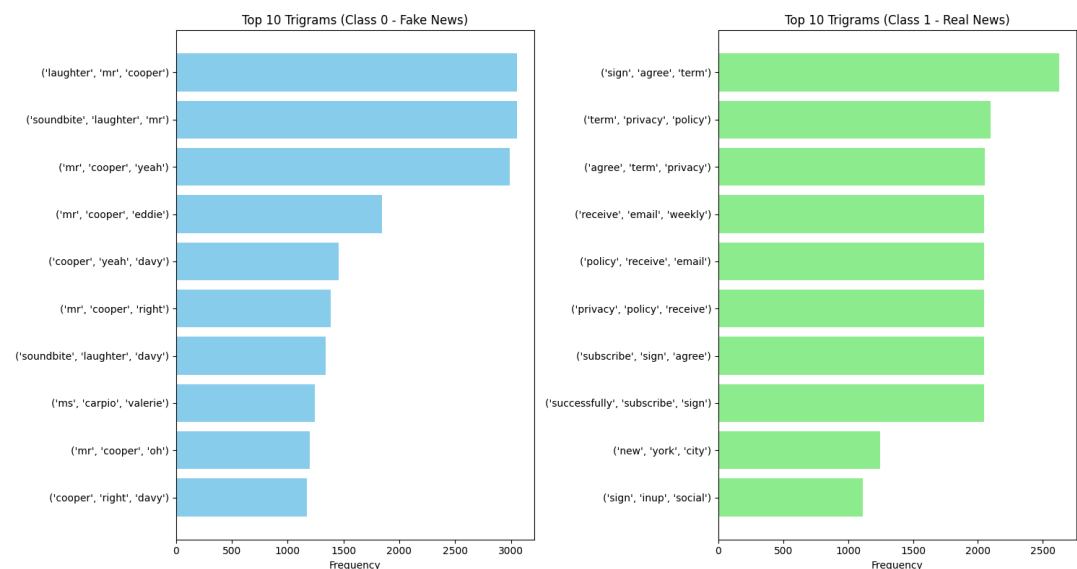


Top-10 Trigrams

```
In [26]: # Filter and plot top-10 trigrams for both fake and real news articles
def get_top_trigrams(text_series):
    all_trigrams = []
    for text in text_series.dropna():
        tokens = word_tokenize(text.lower())
        trigram_list = list(trigrams(tokens))
        all_trigrams.extend(trigram_list)
    return Counter(all_trigrams).most_common(10)

# Filter data for class 0 and class 1
class_0_data = data[data['class'] == 0]['text']
class_1_data = data[data['class'] == 1]['text']
top_trigrams_class_0 = get_top_trigrams(class_0_data)
top_trigrams_class_1 = get_top_trigrams(class_1_data)
trigrams_0, counts_0 = zip(*sorted(top_trigrams_class_0, key=lambda x: x[1]))
trigrams_1, counts_1 = zip(*sorted(top_trigrams_class_1, key=lambda x: x[1]))

# Create subplots
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 8))
# Plot for class 0
ax1.barh([str(tg) for tg in trigrams_0], counts_0, color='skyblue')
ax1.set_title('Top 10 Trigrams (Class 0 - Fake News)')
ax1.set_xlabel('Frequency')
# Plot for class 1
ax2.barh([str(tg) for tg in trigrams_1], counts_1, color='lightgreen')
ax2.set_title('Top 10 Trigrams (Class 1 - Real News)')
ax2.set_xlabel('Frequency')
plt.tight_layout()
plt.savefig('/kaggle/working/images/top-10-trigrams.png', dpi=300)
plt.show()
```



3. Modelling

```
In [28]: ▶ # Create a copy of data
df = data.copy()

# Define endog and exog
endog = df["class"]
exog = df[['text']]

# First train-test split (70-30)
X_temp, X_test, y_temp, y_test = train_test_split(exog, endog, te

# Second split temp into train and validation (70-30 of remaining
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp,
```

```
In [29]: ▶ # Tokenize text feature for Word2Vec
def tokenize_text(text_series):
    return [word_tokenize(text.lower()) for text in text_series.c
tokenized_spacy_text = tokenize_text(exog['text'])

# Train Word2Vec model on the tokenized text
word2vec_model = Word2Vec(sentences=tokenized_spacy_text, vector_

# Function to get average Word2Vec vector for a text
def get_word2vec_vector(text, model, vector_size=100):
    words = word_tokenize(text.lower())
    word_vectors = [model.wv[word] for word in words if word in m
    if len(word_vectors) == 0:
        return np.zeros(vector_size)
    return np.mean(word_vectors, axis=0)

# Vectorize spacy_text using Word2Vec
X_train_1 = np.array([get_word2vec_vector(text, word2vec_model) f
X_val_1 = np.array([get_word2vec_vector(text, word2vec_model) for
X_test_1 = np.array([get_word2vec_vector(text, word2vec_model) fo

# Check shapes
print("X_train shape:", X_train_1.shape)
print("X_val shape:", X_val_1.shape)
print("X_test shape:", X_test_1.shape)
```

```
X_train shape: (11007, 100)
X_val shape: (4718, 100)
X_test shape: (6740, 100)
```

Objective 2: Build a Baseline Model (Logistic Regression)

- The target variable's classes are imbalanced. Two strategies to address class imbalance are incorporated in the modelling pipelines.
 - **SMOTE** is implemented in a pipeline to generate synthetic samples for the minority class.
 - **Random Undersampling** is implemented in a pipeline to reduce entities for the majority class.

```

In [30]: ► # Prepare training and validation sets
X_train_logistic = np.vstack((X_train_1, X_val_1))
y_train_logistic = np.hstack((y_train, y_val))

# Define pipeline for fitting model with imbalanced data
lr_model_imbalanced = LogisticRegression(max_iter=1000, random_state=42)
cv_scores_imbalanced = cross_val_score(lr_model_imbalanced, X_train_logistic, y_train_logistic, cv=5)
print("\n[Pipeline 1: Imbalanced]")
print("Unbalanced X_train_logistic shape:", X_train_logistic.shape)
print("CV Scores:", cv_scores_imbalanced)
print("Average recall:", cv_scores_imbalanced.mean())

# Define pipeline for fitting model with oversampled minority class
smote = SMOTE(sampling_strategy='minority', random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(X_train_logistic, y_train_logistic)
lr_model_smote = LogisticRegression(max_iter=1000, random_state=42)
cv_scores_smote = cross_val_score(lr_model_smote, X_train_smote, y_train_smote, cv=5)
print("\n[Pipeline 2: SMOTE Oversampling]")
print("SMOTE-balanced X_train_logistic shape:", X_train_smote.shape)
print("CV Scores:", cv_scores_smote)
print("Average recall:", cv_scores_smote.mean())

# Define pipeline for fitting model with undersampled majority class
rus = RandomUnderSampler(sampling_strategy='majority', random_state=42)
X_train_rus, y_train_rus = rus.fit_resample(X_train_logistic, y_train_logistic)
lr_model_rus = LogisticRegression(max_iter=1000, random_state=42)
cv_scores_rus = cross_val_score(lr_model_rus, X_train_rus, y_train_rus, cv=5)
print("\n[Pipeline 3: Random Undersampling]")
print("RUS-balanced X_train_logistic shape:", X_train_rus.shape)
print("CV Scores:", cv_scores_rus)
print("Average recall:", cv_scores_rus.mean())

```

```

[Pipeline 1: Imbalanced]
Unbalanced X_train_logistic shape: (15725, 100)
CV Scores: [0.86005195 0.86924916 0.85995898 0.85942808 0.85198408]
Average recall: 0.860134451280202

```

```

[Pipeline 2: SMOTE Oversampling]
SMOTE-balanced X_train_logistic shape: (16378, 100)
CV Scores: [0.86263736 0.87026862 0.85714286 0.86288891 0.85436707]
Average recall: 0.8614609648818569

```

```

[Pipeline 3: Random Undersampling]
RUS-balanced X_train_logistic shape: (15072, 100)
CV Scores: [0.85673092 0.86764529 0.86562707 0.86098208 0.85799602]
Average recall: 0.8617962769674323

```



```

In [31]: ▶ # Predictions by model fitted with imbalanced data
lr_model_imbalanced.fit(X_train_logistic, y_train_logistic)
y_pred_imbalanced = lr_model_imbalanced.predict(X_test_1)
print("\nClassification Report : Logistic Regression fitted on im
print(classification_report(y_test, y_pred_imbalanced, target_name
y_proba_imbalanced = lr_model_imbalanced.predict_proba(X_test_1)[
fpr_imbalanced, tpr_imbalanced, _ = roc_curve(y_test, y_proba_imb
roc_auc_imbalanced = auc(fpr_imbalanced, tpr_imbalanced)

# Predictions by model fitted with SMOTE balanced data
lr_model_smote.fit(X_train_smote, y_train_smote)
y_pred_smote_balanced = lr_model_smote.predict(X_test_1)
print("\nClassification Report : Logistic Regression fitted on SM
print(classification_report(y_test, y_pred_smote_balanced, target_n
y_proba_smote = lr_model_smote.predict_proba(X_test_1)[: , 1]
fpr_smote, tpr_smote, _ = roc_curve(y_test, y_proba_smote)
roc_auc_smote = auc(fpr_smote, tpr_smote)

# Predictions by model fitted with RUS balanced data
lr_model_rus.fit(X_train_rus, y_train_rus)
y_pred_rus_balanced = lr_model_rus.predict(X_test_1) # Fixed bug
print("\nClassification Report : Logistic Regression fitted on RU
print(classification_report(y_test, y_pred_rus_balanced, target_r
y_proba_rus = lr_model_rus.predict_proba(X_test_1)[: , 1]
fpr_rus, tpr_rus, _ = roc_curve(y_test, y_proba_rus)
roc_auc_rus = auc(fpr_rus, tpr_rus)

# Plot Confusion Matrices
fig, axes = plt.subplots(1, 3, figsize=(12, 5))
cm1 = confusion_matrix(y_test, y_pred_imbalanced)
disp1 = ConfusionMatrixDisplay(confusion_matrix=cm1, display_label
disp1.plot(ax=axes[0], cmap='Blues', colorbar=False)
axes[0].set_title("Imbalanced Data")
cm2 = confusion_matrix(y_test, y_pred_smote_balanced)
disp2 = ConfusionMatrixDisplay(confusion_matrix=cm2, display_label
disp2.plot(ax=axes[1], cmap='Blues', colorbar=False)
axes[1].set_title("SMOTE Balanced Data")
cm3 = confusion_matrix(y_test, y_pred_rus_balanced)
disp3 = ConfusionMatrixDisplay(confusion_matrix=cm3, display_label
disp3.plot(ax=axes[2], cmap='Blues', colorbar=False)
axes[2].set_title("RUS Balanced Data")
plt.tight_layout()
plt.savefig('/kaggle/working/images/logistic-regression-confusion
plt.show()

# Plot ROC-AUC curves
plt.figure(figsize=(10, 7))
plt.plot(fpr_imbalanced, tpr_imbalanced, label=f"Imbalanced (AUC
plt.plot(fpr_smote, tpr_smote, label=f"SMOTE Oversampling (AUC =
plt.plot(fpr_rus, tpr_rus, label=f"Random Undersampling (AUC = {r
plt.plot([0, 1], [0, 1], 'k--', linewidth=1)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curves for Logistic Regression with Different Samp
plt.legend(loc="lower right")
plt.grid(True)
plt.tight_layout()
plt.savefig('/kaggle/working/images/logistic-regression-ROC-AUC.p

```

```
plt.show()
```

Classification Report : Logistic Regression fitted on imbalanced data

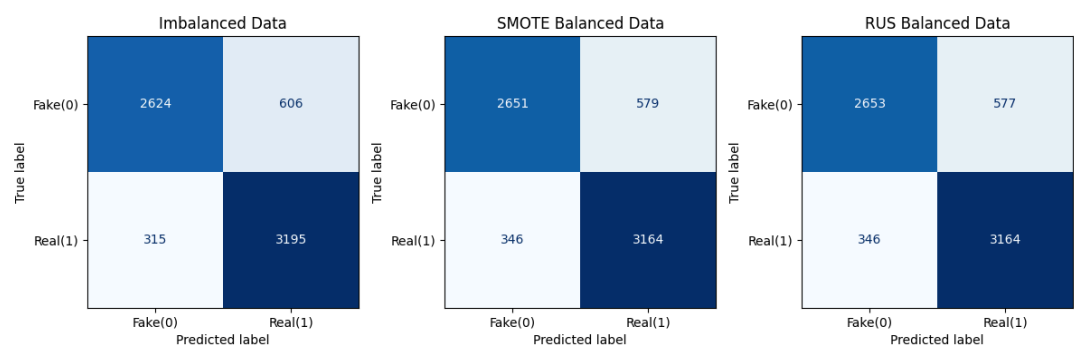
	precision	recall	f1-score	support
Fake(0)	0.8928	0.8124	0.8507	3230
Real (1)	0.8406	0.9103	0.8740	3510
accuracy			0.8634	6740
macro avg	0.8667	0.8613	0.8624	6740
weighted avg	0.8656	0.8634	0.8628	6740

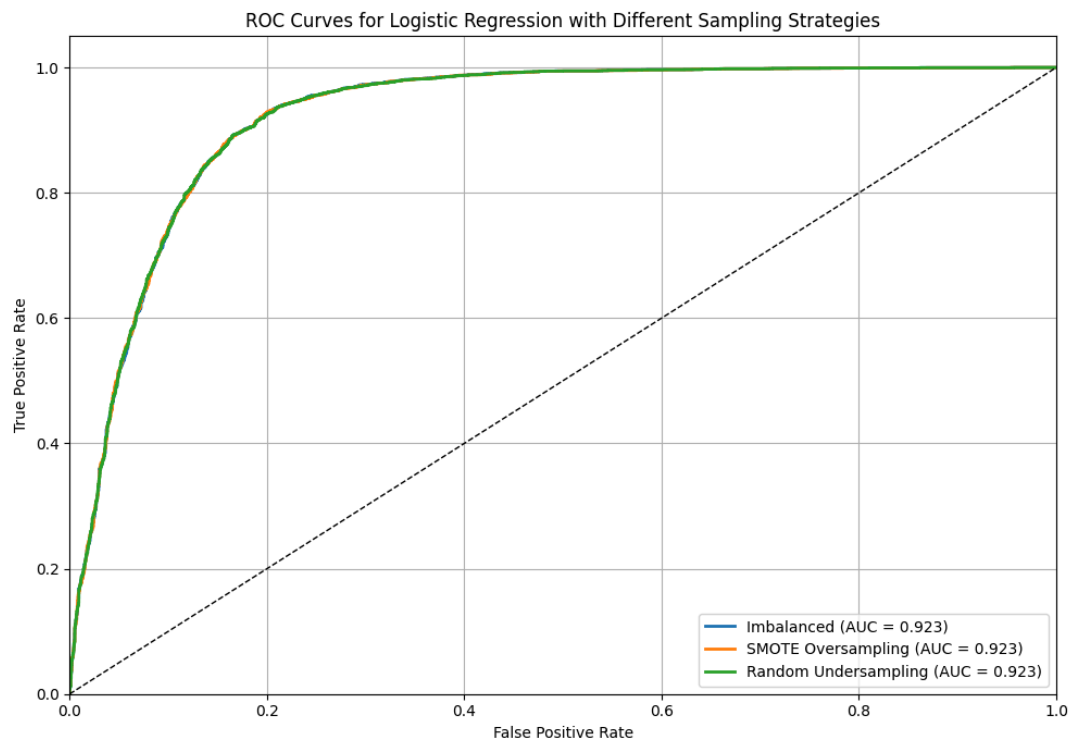
Classification Report : Logistic Regression fitted on SMOTE balanced data

	precision	recall	f1-score	support
Fake(0)	0.8846	0.8207	0.8515	3230
Real (1)	0.8453	0.9014	0.8725	3510
accuracy			0.8628	6740
macro avg	0.8649	0.8611	0.8620	6740
weighted avg	0.8641	0.8628	0.8624	6740

Classification Report : Logistic Regression fitted on RUS balanced data

	precision	recall	f1-score	support
Fake(0)	0.8846	0.8214	0.8518	3230
Real (1)	0.8458	0.9014	0.8727	3510
accuracy			0.8631	6740
macro avg	0.8652	0.8614	0.8623	6740
weighted avg	0.8644	0.8631	0.8627	6740





The baseline model fitted with RUS-balanced data yields better performance since the aim is to catch as many fake news articles as possible.

- Its **recall** score for `class 0 Fake article` is highest at 0.8214.
- Its **macro-averaged recall** is highest at 0.8614.

Objective 3: Build an XGBoost model, LSTM neural network, and Finetune the RoBERTa-base Transformer

XGBoost Model

```
In [32]: ► # Prepare training and validation sets
X_train_xgb = np.vstack((X_train_1, X_val_1))
y_train_xgb = np.hstack((y_train, y_val))

# Define pipeline for fitting model with imbalanced data
xgb_model_imbalanced = XGBClassifier(use_label_encoder=False, eval_metric='log_loss')
cv_scores_imbalanced = cross_val_score(xgb_model_imbalanced, X_train_xgb, y_train_xgb, cv=5)
print("\n[Pipeline 1: Imbalanced]")
print("Unbalanced X_train_xgb shape:", X_train_xgb.shape)
print("CV Scores:", cv_scores_imbalanced)
print("Average recall:", cv_scores_imbalanced.mean())

# Define pipeline for fitting model with oversampled minority class
smote = SMOTE(sampling_strategy='minority', random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(X_train_xgb, y_train_xgb)
xgb_model_smote = XGBClassifier(use_label_encoder=False, eval_metric='log_loss')
cv_scores_smote = cross_val_score(xgb_model_smote, X_train_smote, y_train_smote, cv=5)
print("\n[Pipeline 2: SMOTE Oversampling]")
print("SMOTE-balanced X_train_xgb shape:", X_train_smote.shape)
print("CV Scores:", cv_scores_smote)
print("Average recall:", cv_scores_smote.mean())

# Define pipeline for fitting model with undersampled majority class
rus = RandomUnderSampler(sampling_strategy='majority', random_state=42)
X_train_rus, y_train_rus = rus.fit_resample(X_train_xgb, y_train_xgb)
xgb_model_rus = XGBClassifier(use_label_encoder=False, eval_metric='log_loss')
cv_scores_rus = cross_val_score(xgb_model_rus, X_train_rus, y_train_rus, cv=5)
print("\n[Pipeline 3: Random Undersampling]")
print("RUS-balanced X_train_xgb shape:", X_train_rus.shape)
print("CV Scores:", cv_scores_rus)
print("Average recall:", cv_scores_rus.mean())
```

```
[Pipeline 1: Imbalanced]
Unbalanced X_train_xgb shape: (15725, 100)
CV Scores: [0.87078858 0.87560615 0.87215542 0.87365514 0.867917
23]
Average recall: 0.8720245019182133
```

```
[Pipeline 2: SMOTE Oversampling]
SMOTE-balanced X_train_xgb shape: (16378, 100)
CV Scores: [0.88400488 0.88095238 0.87728938 0.8836532 0.884281
6 ]
Average recall: 0.8820362899165586
```

```
[Pipeline 3: Random Undersampling]
RUS-balanced X_train_xgb shape: (15072, 100)
CV Scores: [0.88392849 0.86831216 0.86629064 0.87093563 0.872594
56]
Average recall: 0.8724122969907013
```



```

In [33]: ▶ # Predictions by model fitted with imbalanced data
xgb_model_imbalanced.fit(X_train_xgb, y_train_xgb)
y_pred_imbalanced = xgb_model_imbalanced.predict(X_test_1)
print("\nClassification Report : XGBoost fitted on imbalanced data")
print(classification_report(y_test, y_pred_imbalanced, target_name='class'))
y_proba_imbalanced = xgb_model_imbalanced.predict_proba(X_test_1)
fpr_imbalanced, tpr_imbalanced, _ = roc_curve(y_test, y_proba_imbalanced)
roc_auc_imbalanced = auc(fpr_imbalanced, tpr_imbalanced)

# Predictions by model fitted with SMOTE balanced data
xgb_model_smote.fit(X_train_smote, y_train_smote)
y_pred_smote_balanced = xgb_model_smote.predict(X_test_1)
print("\nClassification Report : XGBoost fitted on SMOTE balanced data")
print(classification_report(y_test, y_pred_smote_balanced, target_name='class'))
y_proba_smote = xgb_model_smote.predict_proba(X_test_1)[:, 1]
fpr_smote, tpr_smote, _ = roc_curve(y_test, y_proba_smote)
roc_auc_smote = auc(fpr_smote, tpr_smote)

# Predictions by model fitted with RUS balanced data
xgb_model_rus.fit(X_train_rus, y_train_rus)
y_pred_rus_balanced = xgb_model_rus.predict(X_test_1)
print("\nClassification Report : XGBoost fitted on RUS balanced data")
print(classification_report(y_test, y_pred_rus_balanced, target_name='class'))
y_proba_rus = xgb_model_rus.predict_proba(X_test_1)[:, 1]
fpr_rus, tpr_rus, _ = roc_curve(y_test, y_proba_rus)
roc_auc_rus = auc(fpr_rus, tpr_rus)

# Plot Confusion Matrices
fig, axes = plt.subplots(1, 3, figsize=(12, 5))
cm1 = confusion_matrix(y_test, y_pred_imbalanced)
disp1 = ConfusionMatrixDisplay(confusion_matrix=cm1, display_labels=classes_)
disp1.plot(ax=axes[0], cmap='Blues', colorbar=False)
axes[0].set_title("Imbalanced Data")
cm2 = confusion_matrix(y_test, y_pred_smote_balanced)
disp2 = ConfusionMatrixDisplay(confusion_matrix=cm2, display_labels=classes_)
disp2.plot(ax=axes[1], cmap='Blues', colorbar=False)
axes[1].set_title("SMOTE Balanced Data")
cm3 = confusion_matrix(y_test, y_pred_rus_balanced)
disp3 = ConfusionMatrixDisplay(confusion_matrix=cm3, display_labels=classes_)
disp3.plot(ax=axes[2], cmap='Blues', colorbar=False)
axes[2].set_title("RUS Balanced Data")
plt.tight_layout()
plt.savefig('/kaggle/working/images/xgboost-confusion-matrices.png')
plt.show()

# Plot ROC-AUC curves
plt.figure(figsize=(10, 7))
plt.plot(fpr_imbalanced, tpr_imbalanced, label=f"Imbalanced (AUC = {roc_auc_imbalanced:.2f})")
plt.plot(fpr_smote, tpr_smote, label=f"SMOTE Oversampling (AUC = {roc_auc_smote:.2f})")
plt.plot(fpr_rus, tpr_rus, label=f"Random Undersampling (AUC = {roc_auc_rus:.2f})")
plt.plot([0, 1], [0, 1], 'k--', linewidth=1)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curves for XGBoost with Different Sampling Strategies")
plt.legend(loc="lower right")
plt.grid(True)
plt.tight_layout()
plt.savefig('/kaggle/working/images/xgboost-ROC-AUC.png', dpi=300)

```

```
plt.show()
```

Classification Report : XGBoost fitted on imbalanced data

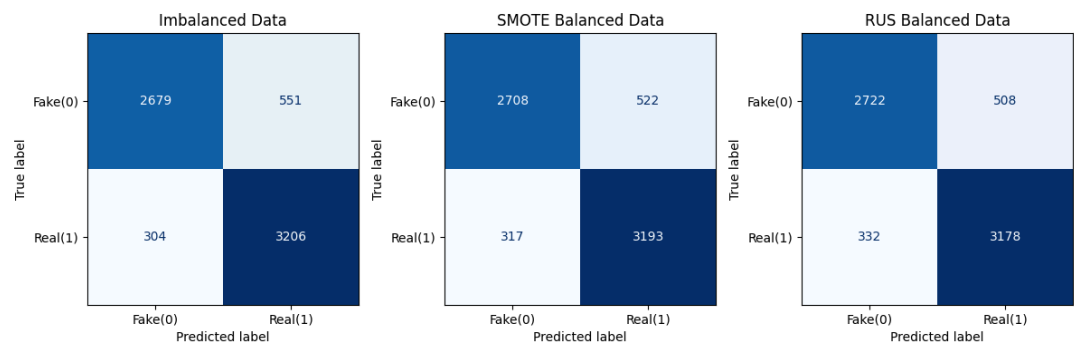
	precision	recall	f1-score	support
Fake(0)	0.8981	0.8294	0.8624	3230
Real (1)	0.8533	0.9134	0.8823	3510
accuracy			0.8731	6740
macro avg	0.8757	0.8714	0.8724	6740
weighted avg	0.8748	0.8731	0.8728	6740

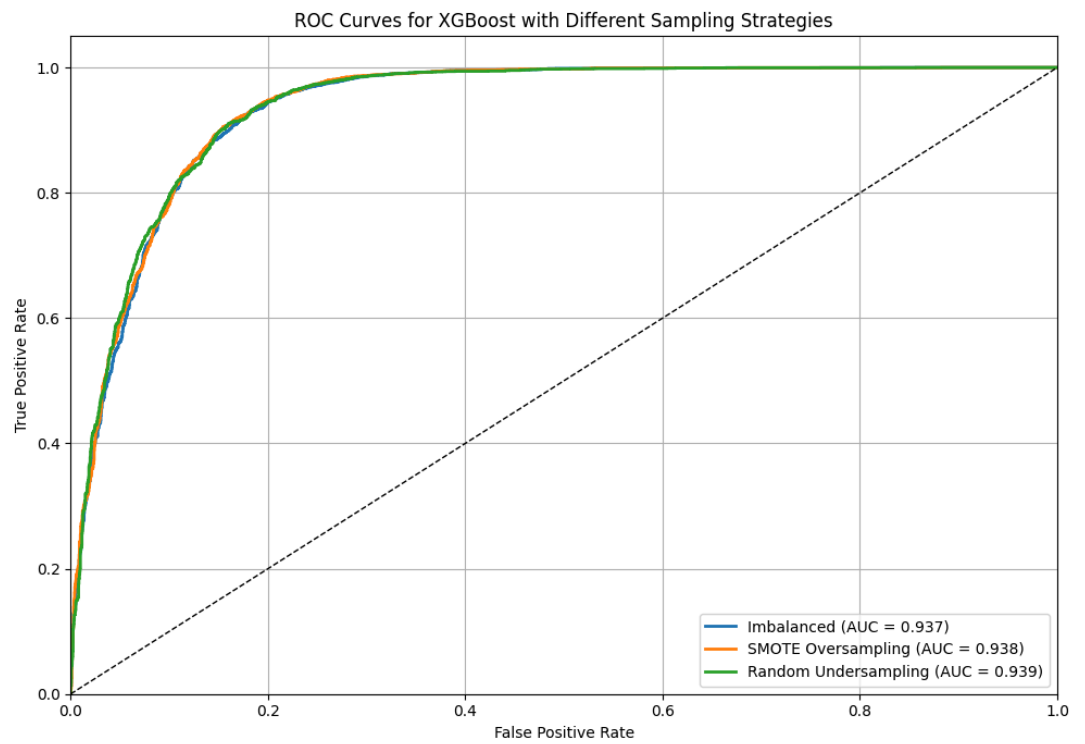
Classification Report : XGBoost fitted on SMOTE balanced data

	precision	recall	f1-score	support
Fake(0)	0.8952	0.8384	0.8659	3230
Real (1)	0.8595	0.9097	0.8839	3510
accuracy			0.8755	6740
macro avg	0.8773	0.8740	0.8749	6740
weighted avg	0.8766	0.8755	0.8752	6740

Classification Report : XGBoost fitted on RUS balanced data

	precision	recall	f1-score	support
Fake(0)	0.8913	0.8427	0.8663	3230
Real (1)	0.8622	0.9054	0.8833	3510
accuracy			0.8754	6740
macro avg	0.8767	0.8741	0.8748	6740
weighted avg	0.8761	0.8754	0.8751	6740





The XGBoost model fitted with RUS-balanced data yields better performance since the aim is to catch as many fake news articles as possible.

- Its **recall** score for `class 0 Fake article`` is highest at 0.8427.
- Its **macro-averaged recall** is highest at 0.8741.

LSTM Neural Network

```

In [47]: ► # Define Parameters
max_len = 300 # Max tokens per sample
embedding_dim = 100 # As set in Word2Vec

# Define function to convert the text feature to vector sequences
def text_to_vector_sequence(text, model, max_len=300, embedding_dim=100):
    tokens = word_tokenize(text.lower())
    sequence = [model.wv[token] for token in tokens if token in model.wv]
    if len(sequence) == 0:
        return np.zeros((max_len, embedding_dim))
    sequence = sequence[:max_len] # Truncate
    if len(sequence) < max_len: # Pad
        padding = np.zeros((max_len - len(sequence), embedding_dim))
        sequence = np.vstack((sequence, padding))
    return np.array(sequence)

# Define function to compute f1-score metric
class F1Score(tf.keras.metrics.Metric):
    def __init__(self, name='f1_score', **kwargs):
        super().__init__(name=name, **kwargs)
        self.precision = Precision()
        self.recall = Recall()
    def update_state(self, y_true, y_pred, sample_weight=None):
        y_pred_binary = tf.round(y_pred)
        self.precision.update_state(y_true, y_pred_binary, sample_weight)
        self.recall.update_state(y_true, y_pred_binary, sample_weight)
    def result(self):
        p = self.precision.result()
        r = self.recall.result()
        return 2 * ((p * r) / (p + r + tf.keras.backend.epsilon()))
    def reset_state(self):
        self.precision.reset_state()
        self.recall.reset_state()

def build_lstm_model(max_len, embedding_dim):
    inputs = Input(shape=(max_len, embedding_dim))
    x = LSTM(64)(inputs)
    x = Dropout(0.3)(x)
    x = Dense(32, activation='relu')(x)
    outputs = Dense(1, activation='sigmoid')(x)
    model = Model(inputs, outputs)
    return model

```



```

In [48]: ► # Define function to train and evaluate the models' respective pe
def train_and_evaluate_pipeline(
    X_train_text, X_val_text, X_test_text,
    y_train, y_val, y_test,
    word2vec_model, max_len, embedding_dim,
    sampling_strategy=None, name="Imbalanced"):
    print(f"\n Training on {name} dataset")

    # Convert the text feature to sequences
    X_train_seq = np.array([text_to_vector_sequence(text, word2vec_model) for text in X_train_text])
    X_val_seq = np.array([text_to_vector_sequence(text, word2vec_model) for text in X_val_text])
    X_test_seq = np.array([text_to_vector_sequence(text, word2vec_model) for text in X_test_text])

    # Reshape for resampling (flatten to 2D)
    if sampling_strategy:
        X_train_flat = X_train_seq.reshape((X_train_seq.shape[0], X_train_seq.shape[-1]))
        X_val_flat = X_val_seq.reshape((X_val_seq.shape[0], X_val_seq.shape[-1]))
        if sampling_strategy == "smote":
            sampler = SMOTE(sampling_strategy='minority', random_state=42)
        elif sampling_strategy == "rus":
            sampler = RandomUnderSampler(sampling_strategy='majority', random_state=42)
        else:
            raise ValueError("sampling_strategy must be 'smote' or 'rus'")
        X_train_resampled, y_train_resampled = sampler.fit_resample(X_train_flat, y_train)
        X_val_resampled, y_val_resampled = sampler.fit_resample(X_val_flat, y_val)
        # Reshape back to 3D
        X_train_seq = X_train_resampled.reshape((-1, max_len, embedding_dim))
        X_val_seq = X_val_resampled.reshape((-1, max_len, embedding_dim))
        y_train_used, y_val_used = y_train_resampled, y_val_resampled
    else:
        y_train_used, y_val_used = y_train, y_val

    # Build and compile model
    model = build_lstm_model(max_len, embedding_dim)
    model.compile(
        optimizer='adam',
        loss='binary_crossentropy',
        metrics=[
            Precision(name='precision'),
            Recall(name='recall'),
            F1Score(name='f1_score')])

    # Train
    history = model.fit(
        X_train_seq, y_train_used,
        validation_data=(X_val_seq, y_val_used),
        epochs=6,
        batch_size=64,
        verbose=1)

    # Extract metrics per epoch
    metrics_df = pd.DataFrame({
        'epoch': range(1, len(history.history['loss']) + 1),
        'train_loss': history.history['loss'],
        'val_loss': history.history['val_loss'],
        'val_precision': history.history['val_precision'],
        'val_recall': history.history['val_recall'],
        'val_f1_score': history.history['f1_score']})
    print(f"\n Metrics per epoch - {name}:")
    print(metrics_df.round(4))

```

```
# Plot Train vs Val Loss plots
plt.figure(figsize=(8, 5))
plt.plot(metrics_df['epoch'], metrics_df['train_loss'], label='Train Loss')
plt.plot(metrics_df['epoch'], metrics_df['val_loss'], label='Val Loss')
plt.title(f'Training vs Validation Loss - {name}')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()

# Save training loss vs. validation loss plots to /kaggle/working
save_name = name.replace(" ", "_").replace("-", "_")
plot_path = os.path.join(images_dir, f"LSTM-loss-curve-({save_name})")
plt.savefig(plot_path)
plt.show()

# Test set prediction
y_test_pred_proba = model.predict(X_test_seq).ravel()
y_test_pred = (y_test_pred_proba > 0.5).astype(int)
return history, metrics_df, model, y_test_pred_proba, y_test_pred
```

```

In [49]: ▶ # Run pipelines
          results = {}

          # Imbalanced
          print("="*60)
          history_imb, df_imb, model_imb, y_test_proba_imb, y_test_pred_imb,
            X_train['text'], X_val['text'], X_test['text'],
            y_train, y_val, y_test,
            word2vec_model, max_len, embedding_dim,
            sampling_strategy=None,
            name="Imbalanced")
          results['Imbalanced'] = (y_test_true, y_test_pred_imb, y_test_proba_imb)

          # SMOTE
          print("="*60)
          history_smote, df_smote, model_smote, y_test_proba_smote, y_test_pred_smote,
            X_train['text'], X_val['text'], X_test['text'],
            y_train, y_val, y_test,
            word2vec_model, max_len, embedding_dim,
            sampling_strategy="smote",
            name="SMOTE Balanced")
          results['SMOTE'] = (y_test_true, y_test_pred_smote, y_test_proba_smote)

          # RUS
          print("="*60)
          history_rus, df_rus, model_rus, y_test_proba_rus, y_test_pred_rus,
            X_train['text'], X_val['text'], X_test['text'],
            y_train, y_val, y_test,
            word2vec_model, max_len, embedding_dim,
            sampling_strategy="rus",
            name="RUS Balanced")
          results['RUS'] = (y_test_true, y_test_pred_rus, y_test_proba_rus)

```

```

=====

Training on Imbalanced dataset
Epoch 1/6
172/172 ————— 9s 34ms/step - f1_score: 0.6366 -
loss: 0.6201 - precision: 0.6769 - recall: 0.6081 - val_f1_score: 0.8508 - val_loss: 0.4327 - val_precision: 0.7635 - val_recall: 0.9605
Epoch 2/6
172/172 ————— 3s 18ms/step - f1_score: 0.7894 -
loss: 0.5082 - precision: 0.7337 - recall: 0.8555 - val_f1_score: 0.4888 - val_loss: 0.6258 - val_precision: 0.6805 - val_recall: 0.3814
Epoch 3/6
172/172 ————— 3s 18ms/step - f1_score: 0.5784 -
loss: 0.6553 - precision: 0.6283 - recall: 0.5370 - val_f1_score: 0.5759 - val_loss: 0.6425 - val_precision: 0.6456 - val_recall: 0.5197
Epoch 4/6
172/172 ————— 3s 18ms/step - f1_score: 0.5784 -
loss: 0.6553 - precision: 0.6283 - recall: 0.5370 - val_f1_score: 0.5759 - val_loss: 0.6425 - val_precision: 0.6456 - val_recall: 0.5197

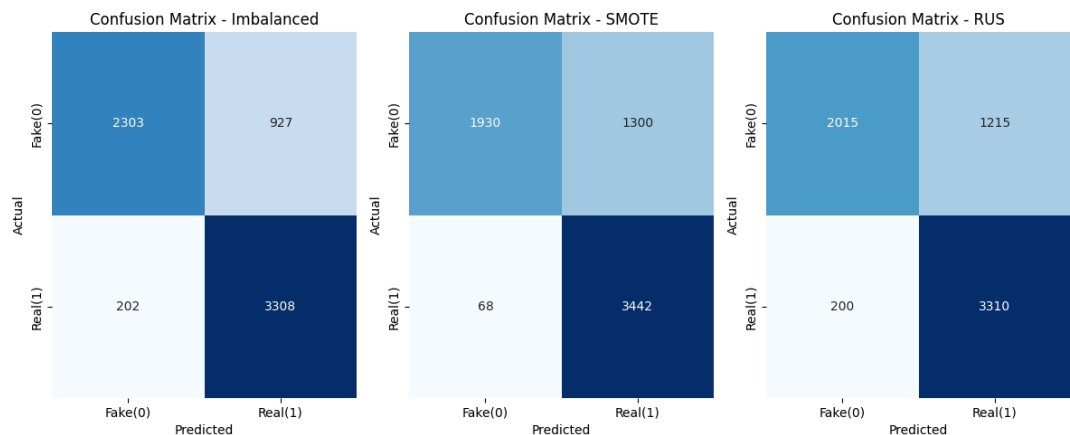
```

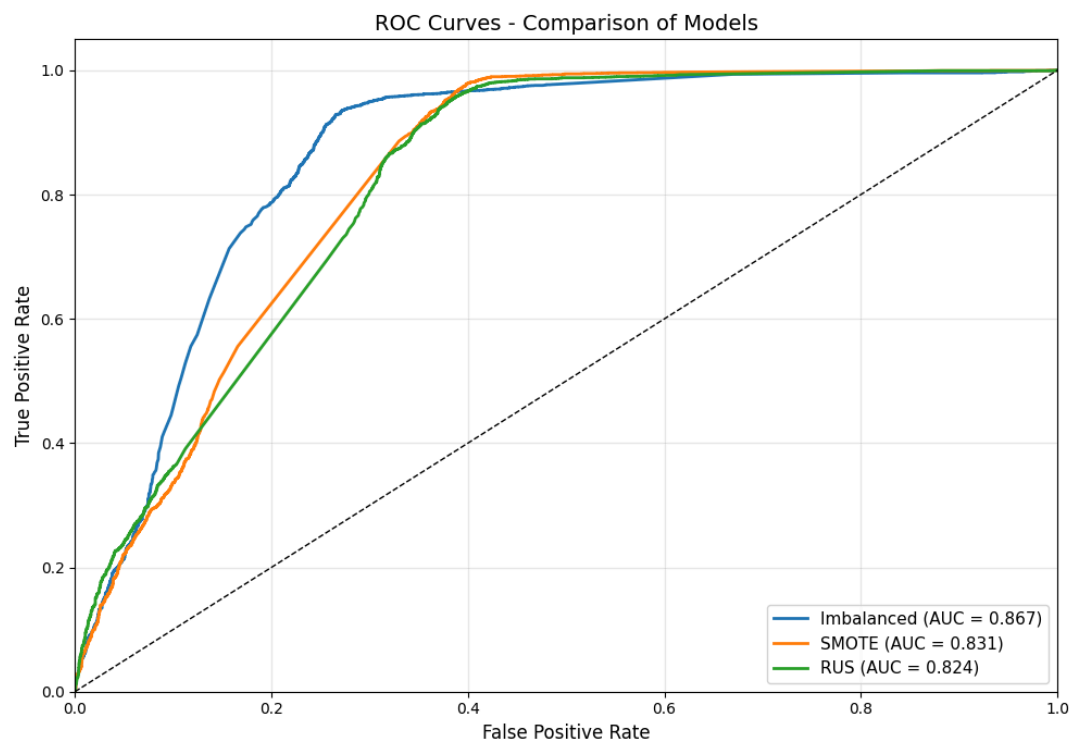
```

In [50]: ▶ # Plot confusion matrices
fig, axes = plt.subplots(1, 3, figsize=(12, 5))
class_names = ['Fake(0)', 'Real(1)']
datasets = ['Imbalanced-LSTM', 'SMOTE-LSTM', 'RUS-LSTM']
for ax, (name, (y_true, y_pred, _)) in zip(axes, results.items()):
    cm = confusion_matrix(y_true, y_pred)
    sns.heatmap(
        cm,
        annot=True,
        fmt='d',
        cmap='Blues',
        ax=ax,
        cbar=False,
        xticklabels=class_names,
        yticklabels=class_names)
    ax.set_title(f'Confusion Matrix - {name}', fontsize=12)
    ax.set_xlabel('Predicted', fontsize=10)
    ax.set_ylabel('Actual', fontsize=10)
plt.tight_layout()
plt.savefig('/kaggle/working/images/LSTM-confusion_matrices.png',
plt.show()

# Plot ROC-AUC curves
plt.figure(figsize=(10, 7))
for name, (y_true, _, y_proba) in results.items():
    fpr, tpr, _ = roc_curve(y_true, y_proba)
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, lw=2, label=f'{name} (AUC = {roc_auc:.3f})')
plt.plot([0, 1], [0, 1], 'k--', linewidth=1)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate', fontsize=12)
plt.ylabel('True Positive Rate', fontsize=12)
plt.title('ROC Curves - Comparison of Models', fontsize=14)
plt.legend(loc="lower right", fontsize=11)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('/kaggle/working/images/LSTM-ROC-AUC.png', dpi=300, b
plt.show()

```





```
In [51]: ▶ # Print per-class and macro/weighted averages
for name, (y_true, y_pred, y_proba) in results.items():
    print(f"\n {name} Model:")
    print("-" * 40)
    print(classification_report(y_true, y_pred, target_names=['Fa
```

Imbalanced Model:

	precision	recall	f1-score	support
Fake(0)	0.9194	0.7130	0.8031	3230
Real(1)	0.7811	0.9425	0.8542	3510
accuracy			0.8325	6740
macro avg	0.8502	0.8277	0.8287	6740
weighted avg	0.8474	0.8325	0.8297	6740

SMOTE Model:

	precision	recall	f1-score	support
Fake(0)	0.9660	0.5975	0.7383	3230
Real(1)	0.7259	0.9806	0.8342	3510
accuracy			0.7970	6740
macro avg	0.8459	0.7891	0.7863	6740
weighted avg	0.8409	0.7970	0.7883	6740

RUS Model:

	precision	recall	f1-score	support
Fake(0)	0.9097	0.6238	0.7401	3230
Real(1)	0.7315	0.9430	0.8239	3510
accuracy			0.7901	6740
macro avg	0.8206	0.7834	0.7820	6740
weighted avg	0.8169	0.7901	0.7838	6740

The LSTM model fitted with imbalanced data yields better performance since the aim is to catch as many fake news articles as possible.

- Its **recall** for class 0 **Fake article** is 0.7130
- Its **macro-averaged recall** is highest at 0.8277

Finetune the RoBERTa-base Transformer

```
In [52]: ▶ # Set up device, pretrained transformer, and tokenizer

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

model_name = "roberta-base" # Set up pretrained transformer
roberta_tokenizer = RobertaTokenizer.from_pretrained(model_name)

Using device: cuda
```

In [53]:  *# Initialize the NewsDataset with text chunking using a sliding window*

```
class NewsDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_length=512,
                 stride=128):
        self.tokenizer = tokenizer
        self.max_length = max_length
        self.stride = stride
        self.all_chunk_encodings = []      # Store {'input_ids',
        self.all_chunk_labels = []        # Store the label for
        self.chunk_original_indices = []   # Store the index of t

    # Process each original text to create chunks
    for original_idx, text in enumerate(texts):
        tokenized_output = tokenizer(
            text,
            truncation=False,
            padding=False,
            max_length=self.max_length,
            stride=self.stride,
            return_overflowing_tokens=True,
            return_tensors="pt")          # Return PyTorch tensors

    # Iterate through each chunk generated for the current original text
    for i in range(len(tokenized_output['input_ids'])):
        chunk_ids = tokenized_output['input_ids'][i]
        chunk_mask = tokenized_output['attention_mask'][i]

        if chunk_ids.shape[0] > max_length:      # Truncate
            chunk_ids = chunk_ids[:max_length]
            chunk_mask = chunk_mask[:max_length]
        elif chunk_ids.shape[0] < max_length:    # Pad
            pad_len = max_length - chunk_ids.shape[0]
            pad_tensor = torch.full((pad_len,), tokenizer.pad_token_id)
            mask_pad = torch.zeros(pad_len, dtype=torch.float)
            chunk_ids = torch.cat([chunk_ids, pad_tensor])
            chunk_mask = torch.cat([chunk_mask, mask_pad])

        self.all_chunk_encodings.append({
            'input_ids': chunk_ids,
            'attention_mask': chunk_mask})
        self.all_chunk_labels.append(labels[original_idx])
        self.chunk_original_indices.append(original_idx)

    # Return the total number of chunks across all original articles
    def __len__(self):
        return len(self.all_chunk_labels)

    # Retrieve a single chunk's data and its associated original index
    def __getitem__(self, idx):
        item = {}
        for key, val in self.all_chunk_encodings[idx].items():
            item[key] = val.clone()
        item['labels'] = torch.tensor(self.all_chunk_labels[idx])
        item['original_index'] = torch.tensor(self.chunk_original_indices[idx])
        return item
```

In [54]: **▶** *# Define function to resample imbalanced data before chunking*

```
def resample_data(X_texts, y_labels, strategy="smote"):
    X_list = X_texts.tolist()
    y_array = np.array(y_labels)

    if strategy == "smote": # Oversample the minority class

        # Use dummy features (lengths) to simulate feature space
        X_dummy = np.array([[len(text)] for text in X_list])
        sampler = SMOTE(sampling_strategy='minority', random_state=None)
        _, y_resampled = sampler.fit_resample(X_dummy, y_array)

        # Resample texts by oversampling the minority class
        from collections import Counter
        counter = Counter(y_array)
        target_count = max(counter.values())
        X_resampled, y_resampled = [], []
        for label in [0, 1]:
            indices = np.where(y_array == label)[0]
            resampled_indices = np.random.choice(indices, size=target_count - counter[label])
            X_resampled.extend([X_list[i] for i in resampled_indices])
            y_resampled.extend([y_array[i] for i in resampled_indices])

    elif strategy == "rus": # Undersample the majority class
        X_dummy = np.array([[len(text)] for text in X_list])
        sampler = RandomUnderSampler(sampling_strategy='majority', random_state=None)
        _, y_resampled = sampler.fit_resample(X_dummy, y_array)

        # Resample texts by undersampling the majority class according to the minority class
        from collections import Counter
        counter = Counter(y_array)
        target_count = min(counter.values())
        X_resampled, y_resampled = [], []
        for label in [0, 1]:
            indices = np.where(y_array == label)[0]
            resampled_indices = np.random.choice(indices, size=target_count)
            X_resampled.extend([X_list[i] for i in resampled_indices])
            y_resampled.extend([y_array[i] for i in resampled_indices])

    else: # imbalanced dataset
        X_resampled, y_resampled = X_list, y_array

    return X_resampled, y_resampled
```



```

In [55]: # Initialize the MetricsTracker class
class MetricsTracker:
    def __init__(self):
        self.epoch_data = []
    def log_epoch(self, epoch, metrics):
        self.epoch_data.append({'epoch': int(epoch), **metrics})
    def get_dataframe(self):
        return pd.DataFrame(self.epoch_data).set_index('epoch')
metrics_tracker = MetricsTracker()

# Define function to compute performance metrics
def compute_metrics(p):
    preds = np.argmax(p.predictions, axis=1)
    probs = torch.softmax(torch.tensor(p.predictions), dim=1).numpy()
    return {
        'accuracy': accuracy_score(p.label_ids, preds),
        'precision': precision_score(p.label_ids, preds, zero_division=0),
        'recall': recall_score(p.label_ids, preds, zero_division=0),
        'f1': f1_score(p.label_ids, preds, zero_division=0),
        'roc_auc': roc_auc_score(p.label_ids, probs)}

# Initialize the CustomTrainer class for logging
class CustomTrainer(Trainer):
    def evaluate(self, *args, **kwargs):
        metrics = super().evaluate(*args, **kwargs)
        self.control = self.callback_handler.on_evaluate(self.args, metrics)
        metrics_tracker.log_epoch(self.state.epoch, metrics)
        return metrics

# Define function to extract average training loss per epoch from log history
def extract_per_epoch_train_loss(trainer):
    train_loss_epochs = {}
    for log in trainer.state.log_history:
        if 'loss' in log and 'epoch' in log:
            epoch = round(log['epoch']) # Round float epoch to int
            if epoch not in train_loss_epochs:
                train_loss_epochs[epoch] = []
            train_loss_epochs[epoch].append(log['loss'])
    # Return average loss per epoch
    return pd.Series({epoch: np.mean(losses) for epoch, losses in train_loss_epochs.items()})

# Define training arguments function
def get_training_args(run_name):
    return TrainingArguments(
        output_dir=f"./roberta_results_{run_name}",
        run_name=f"roberta-{run_name}",
        num_train_epochs=4,
        per_device_train_batch_size=16,
        per_device_eval_batch_size=32,
        learning_rate=2e-5,
        warmup_ratio=0.06,
        weight_decay=0.01,
        eval_strategy="epoch",
        save_strategy="epoch",
        load_best_model_at_end=True,
        metric_for_best_model="recall",
        greater_is_better=True,
        logging_dir=f"./roberta_logs_{run_name}",
        logging_steps=100,
        seed=42,
        fp16=torch.cuda.is_available(),

```

```
report_to="none",  
save_total_limit=2)
```


In [56]: **# Define function to implement finetuning pipelines**

```
def finetune_roberta_pipeline(X_train, y_train, X_val, y_val, X_test, y_test, name):
    print(f"\n Fine-tuning RoBERTa ({name})")
    X_train_res, y_train_res = resample_data(X_train, y_train, stateful=True)

    # Create chunked datasets
    train_dataset = NewsDataset(X_train_res, y_train_res, roberta_tokenizer)
    val_dataset = NewsDataset(X_val.tolist(), y_val.tolist(), roberta_tokenizer)
    test_dataset = NewsDataset(X_test.tolist(), y_test.tolist(), roberta_tokenizer)

    # Load model
    model = RobertaForSequenceClassification.from_pretrained(
        model_name,
        num_labels=2,
        hidden_dropout_prob=0.2,
        attention_probs_dropout_prob=0.2
    ).to(device)

    # Reset tracker and trainer
    global metrics_tracker
    metrics_tracker = MetricsTracker()
    trainer_args = get_training_args(name.replace(" ", "_"))
    trainer = CustomTrainer(
        model=model,
        args=trainer_args,
        train_dataset=train_dataset,
        eval_dataset=val_dataset,
        compute_metrics=compute_metrics,)

    # Train and compile the performance metrics DataFrame
    trainer.train()
    avg_train_loss = extract_per_epoch_train_loss(trainer)
    metrics_df = metrics_tracker.get_dataframe()
    metrics_df = metrics_df.assign(
        train_loss=metrics_df.index.map(avg_train_loss), # Map average train loss to index
        val_loss=metrics_df['eval_loss'])
    print(f"\nMetrics per epoch - {name}:")
    print(metrics_df[['train_loss', 'val_loss', 'eval_precision', 'eval_recall', 'eval_f1_score']])

    # Plot and Save Loss Curve
    plt.figure(figsize=(8, 5))
    epochs = metrics_df.index
    plt.plot(epochs, metrics_df['train_loss'], label='Train Loss')
    plt.plot(epochs, metrics_df['val_loss'], label='Validation Loss')
    plt.title(f'Training vs Validation Loss - RoBERTa ({name})')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plot_path = f"/kaggle/working/images/roberta_loss_{name.replace(' ', '_')}.png"
    plt.savefig(plot_path)
    plt.show()

    # Evaluate respective performance on the test set
    test_outs = trainer.predict(test_dataset)
    y_test_pred = np.argmax(test_outs.predictions, axis=1)
    y_test_proba = torch.softmax(torch.tensor(test_outs.predictions), dim=-1)

    return metrics_df, model, y_test_pred, y_test_proba, y_test
```

```
In [57]: ▶ # Run Pipelines
results = {}

# Imbalanced
print("="*60)
df_imb, model_imb, y_test_pred_imb, y_test_proba_imb, y_test_true = finetune(
    X_train['text'], y_train, X_val['text'], y_val, X_test['text'],
    strategy=None, name="Imbalanced")
results['RoBERTa-Imbalanced'] = (y_test_true, y_test_pred_imb, y_test_proba_imb)

# SMOTE
print("="*60)
df_smote, model_smote, y_test_pred_smote, y_test_proba_smote, _ = finetune(
    X_train['text'], y_train, X_val['text'], y_val, X_test['text'],
    strategy="smote", name="SMOTE")
results['RoBERTa-SMOTE'] = (y_test_true, y_test_pred_smote, y_test_proba_smote)

# RUS
print("="*60)
df_rus, model_rus, y_test_pred_rus, y_test_proba_rus, _ = finetune(
    X_train['text'], y_train, X_val['text'], y_val, X_test['text'],
    strategy="rus", name="RUS")
results['RoBERTa-RUS'] = (y_test_true, y_test_pred_rus, y_test_proba_rus)
```

=====

Fine-tuning RoBERTa (Imbalanced)

Some weights of RobertaForSequenceClassification were not initialized from the model checkpoint at roberta-base and are newly initialized: ['classifier.dense.bias', 'classifier.dense.weight', 'classifier.out_proj.bias', 'classifier.out_proj.weight']

You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

[1376/1376 44:32, Epoch 4/4]

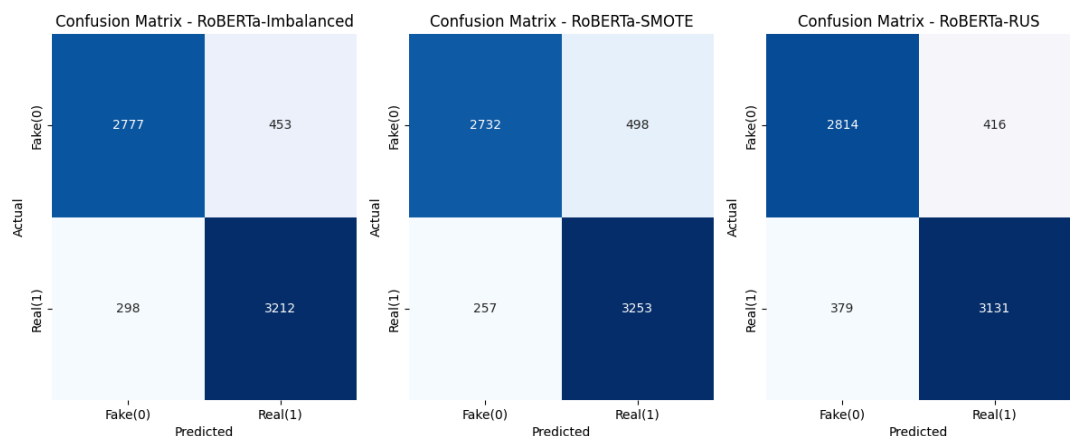
Epoch	Training Loss	Validation Loss	Accuracy	Precision	Recall	F1	Roc Auc
1	0.333100	0.357329	0.856507	0.878080	0.841270	0.859281	0.933772
2	0.281400	0.300859	0.879398	0.878508	0.891738	0.885074	0.946386

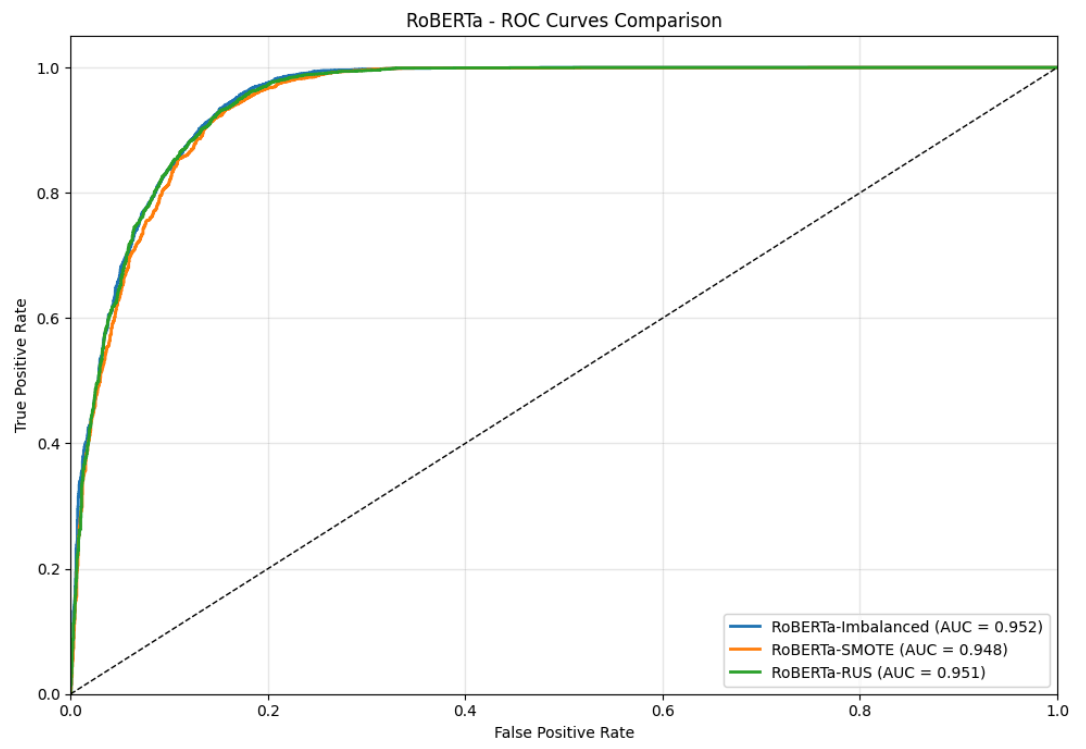
```

In [58]: ▶ # Plot Confusion Matrices
fig, axes = plt.subplots(1, 3, figsize=(12, 5))
class_names = ['Fake(0)', 'Real(1)']
for ax, (name, (y_true, y_pred, _)) in zip(axes, results.items()):
    cm = confusion_matrix(y_true, y_pred)
    sns.heatmap(
        cm,
        annot=True,
        fmt='d',
        cmap='Blues',
        ax=ax,
        cbar=False,
        xticklabels=class_names,
        yticklabels=class_names)
    ax.set_title(f'Confusion Matrix - {name}', fontsize=12)
    ax.set_xlabel('Predicted', fontsize=10)
    ax.set_ylabel('Actual', fontsize=10)
plt.tight_layout()
plt.savefig('/kaggle/working/images/roberta-confusion-matrices.png')
plt.show()

# Plot ROC-AUC curve plots
plt.figure(figsize=(10, 7))
for name, (y_true, _, y_proba) in results.items():
    fpr, tpr, _ = roc_curve(y_true, y_proba)
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, lw=2, label=f'{name} (AUC = {roc_auc:.3f})')
plt.plot([0, 1], [0, 1], 'k--', linewidth=1)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('RoBERTa - ROC Curves Comparison')
plt.legend(loc="lower right")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('/kaggle/working/images/roberta-roc-auc.png', dpi=300)
plt.show()

```





```
In [59]: ▶ # Print per-class and macro/weighted averages
for name, (y_true, y_pred, y_proba) in results.items():
    print(f"\n {name} Model:")
    print("-" * 40)
    print(classification_report(y_true, y_pred, target_names=['Fa
```

RoBERTa-Imbalanced Model:

	precision	recall	f1-score	support
Fake(0)	0.9031	0.8598	0.8809	3230
Real(1)	0.8764	0.9151	0.8953	3510
accuracy			0.8886	6740
macro avg	0.8897	0.8874	0.8881	6740
weighted avg	0.8892	0.8886	0.8884	6740

RoBERTa-SMOTE Model:

	precision	recall	f1-score	support
Fake(0)	0.9140	0.8458	0.8786	3230
Real(1)	0.8672	0.9268	0.8960	3510
accuracy			0.8880	6740
macro avg	0.8906	0.8863	0.8873	6740
weighted avg	0.8897	0.8880	0.8877	6740

RoBERTa-RUS Model:

	precision	recall	f1-score	support
Fake(0)	0.8813	0.8712	0.8762	3230
Real(1)	0.8827	0.8920	0.8873	3510
accuracy			0.8820	6740
macro avg	0.8820	0.8816	0.8818	6740
weighted avg	0.8820	0.8820	0.8820	6740

The findings justify that balancing the dataset used for finetuning pretrained transformers improves performance on the minority class.

- The RoBERTa transformer finetuned with SMOTE-balanced data **RoBERTa-SMOTE** scores a recall of **0.6686** for the minority class Fake(0) and achieves the highest macro-averaged recall (**0.7660**) indicating that the model performs best in terms of capturing both classes equally well.
- The RoBERTa transformer finetuned with RUS-balanced data **RoBERTa-RUS** scores a recall of **0.5167** for the minority class Fake(0) and a macro-averaged recall of **0.6920**.
- The RoBERTa transformer finetuned with imbalanced data **RoBERTa-Imbalanced** achieves the lowest recall of **0.3922** and a macro-averaged recall (**0.6819**).

```
In [60]: # Save Fine-tuned Models
import os
import shutil

# Base directory to save models
models_save_dir = "/kaggle/working/roberta_models_tuned"
os.makedirs(models_save_dir, exist_ok=True)
models_to_save = {
    "roberta_imbalanced": model_imb,
    "roberta_smote": model_smote,
    "roberta_rus": model_rus}

print("Saving fine-tuned RoBERTa models")

for model_name, model in models_to_save.items():
    model_path = os.path.join(models_save_dir, model_name)

    # Remove old directory if exists
    if os.path.exists(model_path):
        shutil.rmtree(model_path)

    # Save model and tokenizer
    model.save_pretrained(model_path)
    roberta_tokenizer.save_pretrained(model_path)
    print(f"Model saved to: {model_path}")
```

Saving fine-tuned RoBERTa models

Model saved to: /kaggle/working/roberta_models_tuned/roberta_imbalanced

Model saved to: /kaggle/working/roberta_models_tuned/roberta_smote

Model saved to: /kaggle/working/roberta_models_tuned/roberta_rus

4. Evaluation and Model Interpretability

Objective 4: Interpret best performing ML model using the LIME library

Best Performing Model

MODEL	Macro-averaged Recall	Recall (Fake)	Recall (Real)	Precision (Fake)	Precision (Real)	Macro-averaged f1	Accuracy	AUC
Logistic Regression-Imbalanced	0.8613	0.8124	0.9103	0.8923	0.8406	0.8624	0.8634	0.930
Logistic Regression-SMOTE	0.8611	0.8207	0.9014	0.8846	0.8453	0.8620	0.8628	0.930
Logistic Regression-RUS	0.8614	0.8214	0.9014	0.8846	0.8458	0.8623	0.8631	0.930
XGBoost-Imbalanced	0.8714	0.8294	0.9134	0.8981	0.8533	0.8724	0.8731	0.937
XGBoost-SMOTE	0.8740	0.8384	0.9097	0.8952	0.8595	0.8749	0.8755	0.938

MODEL	Macro-averaged Recall	Recall (Fake)	Recall (Real)	Precision (Fake)	Precision (Real)	Macro-averaged f1	Accuracy	AUC
XGBoost-RUS	0.8741	0.8427	0.9054	0.8913	0.8622	0.8748	0.8754	0.939
LSTM-Imbalanced	0.8277	0.7130	0.9425	0.9194	0.7811	0.8287	0.8325	0.867
LSTM-SMOTE	0.7891	0.5975	0.9806	0.9660	0.7259	0.7863	0.7970	0.831
LSTM-RUS	0.7834	0.6238	0.9430	0.9097	0.7315	0.7901	0.7901	0.824
RoBERTa-Imbalanced	0.8874	0.8598	0.9151	0.9031	0.8764	0.8881	0.8886	0.952
RoBERTa-SMOTE	0.8863	0.8458	0.9268	0.9140	0.8672	0.8873	0.8880	0.948

The Finetuned RoBERTa model using RUS-balanced dataset is chosen as the better performing model because:

- It achieves the highest **Recall (Fake) (0.8712)** : Highest **true positives for fake (class 0) predictions (2814)**.
- It achieves the highest **Precision (Real) (0.8827)** : Highest certainty (88.27%) for class 1 predictions when an article is actually **real**.
- RoBERTa's contextual understanding from pretraining and fine-tuning with balanced data enhances its robustness, and generalizability.
- The transformer trained with a RUS-balanced dataset outperforms RoBERTa-SMOTE because undersampling the majority class preserves the integrity of the original data.

Interpret the Finetuned RoBERTa-base Model

We utilize the LIME (Local Interpretable Model-agnostic Explanations) library to interpret the predictions of a fine-tuned RoBERTa model, which classifies news articles as either class 0 (fake) or class 1 (real). We load the pre-trained RoBERTa model and its tokenizer from a specified path, setting the model to evaluation mode to disable gradient computation. A predictor function tokenizes input texts, processes them through the model, and applies a softmax function to convert logits into probability scores for the two classes.

The LIME explainer is initialized with class names 'real' and 'fake', enabling local interpretation of a selected news article (e.g., `data.iloc[99, 10]`). It generates explanations by perturbing the text and evaluating the model's predictions on these variations, using 500 samples and highlighting the top 10 features. The results are visualized as a 1x2 horizontal bar plot, with one subplot per class. Bars represent feature weights:

- Green for positive influence (supporting the class).
- Red for negative influence.

The bars are sorted by absolute weight to emphasize the most impactful words. Leveraging the LIME library provides insight into which terms drive the model's classification, enhancing interpretability of the fake/real news prediction.

```
In [63]: ► # Preview a random sample  
data.iloc[99, 8]
```

```
Out[63]: 'angelina jolie engagement ring compare brad pitt give jennifer  
aniston manage account angelina jolie brad pitt engagement annou  
nce week buzz ring ring ring designer robert procop pitt collabo  
rate year order right long hear brangelina pitt famously pop que  
stion wife jennifer aniston make wonder ring help create stack s  
ee friday jolie ring feature massive giant emerald cut stone set  
platinum stone encircle finger vary report price bauble range mi  
llion hard determine cost know carat count couple share engageme  
nt announcement know courtesy jeweler procop diamond high gem qu  
ality jewelry expert michael oconnor estimate carat ring tell we  
ekly emerald cut diamond few facet sparkle cut mean need good qu  
ality light return diamond like pure white pitt ask aniston marr  
y silvia damiani design special sparkler appear small jolie acto  
r girlfriend story begin casually brad pitt like pair earring de  
sign win diamond international award damiani tell business today  
request special item engagement jennifer aniston base model cust  
om design symbol eternity feature round stone surround row spira  
l diamond time value designer later create pitt aniston wedding  
band white gold diamond engrave aniston say brad pitt say jen ri  
ng stunner certainly different aniston classic feel pitt reporte  
dly want look like heart symbolize everlasting love row diamond  
cant miss sparkle hand jolie look line modern architecture enthu  
siast pitt style large perfectly cut diamond centerpiece stone g  
radually decrease size crown like shape oconner note cut gem mut  
e brilliance diamond attention grab shimmer aniston course forge  
t pitt previously engage gwyneth paltrow date bit experience com  
e engagement ring ring like jolie aniston weigh celebrity featur  
e yahoo brad pitt co star talk parenting skill angelina jolie un  
faze oscar leg craze angelina jolie kid happy sinister maleficen  
t role follow omg twitter'
```



```

In [70]: ▶ # Load the fine-tuned model and tokenizer
model_path = "/kaggle/working/roberta_models_tuned/roberta_rus"
tokenizer = RobertaTokenizerFast.from_pretrained(model_path)
model = RobertaForSequenceClassification.from_pretrained(model_path)
model.eval()

# Define the LIME-compatible predictor function
def predictor(texts):
    encodings = tokenizer(texts, truncation=True, padding=True, return_tensors='pt')
    with torch.no_grad():
        outputs = model(**encodings)
    probas = torch.nn.functional.softmax(outputs.logits, dim=-1).cpu().numpy()
    return probas

# LIME explainer instance
class_names = ['Fake Article', 'Real Article']
explainer = LimeTextExplainer(class_names=class_names)

# Choose an article to explain
sample_text = data.iloc[99, 8]

# Get prediction
probas = predictor([sample_text])
predicted_label = int(np.argmax(probas)) # 0 or 1
predicted_class_name = class_names[predicted_label]

# Generate explanation for the predicted label
explanation = explainer.explain_instance(
    sample_text,
    classifier_fn=predictor,
    num_features=10,
    num_samples=500,
    labels=[predicted_label])

# Extract top-10 features for the predicted label
features = explanation.as_list(label=predicted_label)
features_sorted = sorted(features, key=lambda x: abs(x[1]), reverse=True)
features, weights = zip(*features_sorted[:10])
print(f"Predicted Class: {predicted_class_name} (Label {predicted_label})")

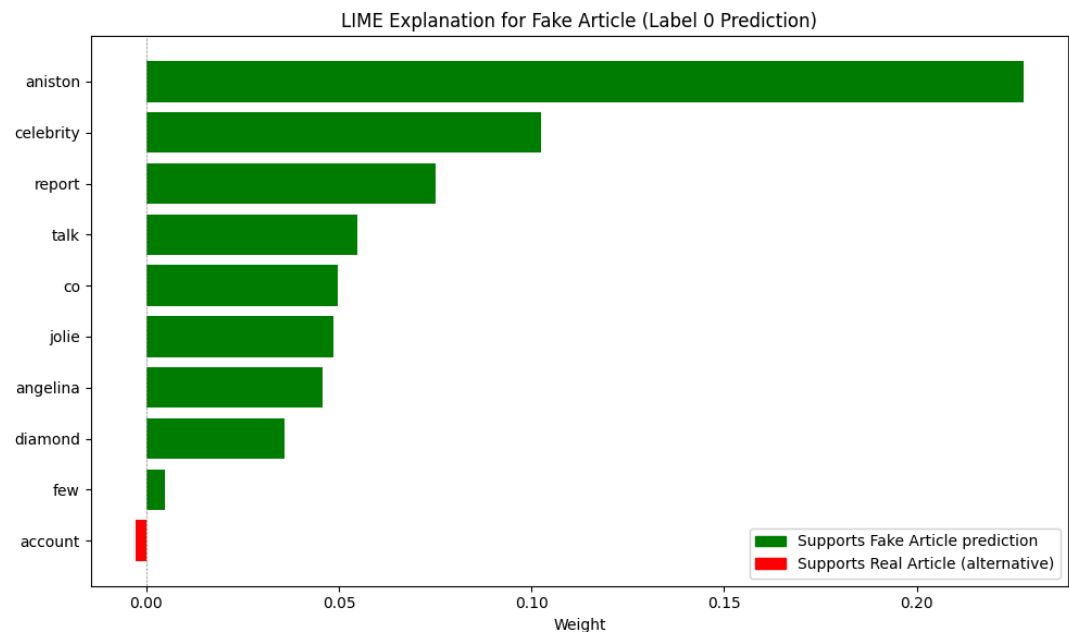
# Plot a horizontal bar plot to visualize top-10 important features
y_pos = np.arange(len(features))
colors = ['green' if w > 0 else 'red' for w in weights]
plt.figure(figsize=(10, 6))
plt.barh(y_pos, weights, align='center', color=colors)
plt.yticks(y_pos, features)
plt.gca().invert_yaxis()
plt.xlabel('Weight')
plt.axvline(x=0, color='gray', linestyle='--', linewidth=0.5)
plt.title(f"LIME Explanation for {predicted_class_name} (Label {predicted_label})")

# Define legend patches
if predicted_label == 0:
    green_label = "Supports Fake Article prediction"
    red_label = "Supports Real Article (alternative)"
else:
    green_label = "Supports Real Article prediction"
    red_label = "Supports Fake Article (alternative)"
green_patch = mpatches.Patch(color='green', label=green_label)
red_patch = mpatches.Patch(color='red', label=red_label)
plt.legend(handles=[green_patch, red_patch], loc='lower right')

```

```
plt.tight_layout()
plt.savefig('./images/lime_explanation_barplot.png', dpi=300, bbox_inches='tight')
plt.show()
```

Predicted Class: Fake Article (Label 0)



5. Deployment

Objective 5: Deploy selected model using FastAPI and Streamlit

Thresholding Strategy

An asymmetric thresholding strategy is adopted to further improve the model's **recall** at the expense of a slight drop in **precision** as follows:

- The threshold for predicting Fake Articles is lowered to **0.45**.
- The threshold for predicting Real Articles is raised to **0.60**.
- Probabilities between (**0.40 to 0.45**) for **Label 0** and (**0.55 to 0.60**) for **Label 1** are flagged for human review.

Containerization Strategy

We leveraged FASTAPI, Streamlit, and Docker to deploy the fine-tuned RoBERTa Transformer Model for fake news detection.

- **Back-end** : The FastAPI app (**main.py**) is responsible for model inference. It loads the saved roberta_model, incorporates the SpaCy text preprocessing pipeline, scrapes article text from a provided URL using BeautifulSoup, predicts whether it is fake or real, and generates LIME explanation results.
- **Front-end** : The Streamlit app (**app.py**) provides an interactive frontend web interface that prompts the user to enter a news article URL for prediction. It sends the URL to the FastAPI endpoint (/predict/) via localhost, displays the article title, prediction label, and generates a LIME explanation plot.
- **Docker** : Used to containerize the front-end and the back-end using two Dockerfiles (one for each).

- The required Python dependencies for the front-end include: streamlit, requests, and matplotlib.
- The required Python dependencies for the back-end include: fastapi, uvicorn, transformers, torch, numpy, lime, pydantic, requests, beautifulsoup4, spacy, and lxml.
- The docker-compose.yml exposes ports: **8000:8000** to the back-end and ports **8501:8501** to the front-end.

6. Conclusion, Recommendations, and Next Steps

6.1 Conclusion

This project successfully developed a robust pipeline for fake news detection. It leveraged advanced NLP techniques and multiple machine learning approaches. The dataset (22,465 news articles) was meticulously processed through feature engineering (domain extraction, text length analysis, preprocessing) and exploratory analysis. The **Recall (Fake)** metric was selected as the success criteria because the project aims to catch as many fake news articles as possible. The Key objectives were addressed by:

- Utilizing the spaCy library to preprocess text data and perform EDA to unveil underlying characteristics among fake and real news articles.
- Building a baseline **Logistic Regression** model to establish initial performance benchmarks.
- Building an ensemble model (**XGBoost**), a neural network (**LSTM**), and finetuning the **RoBERTa-base** transformer.
- Evaluating the models' respective performance on test set and interpreting the alternative with the highest **Recall (Fake)** using LIME.
- Deploying the selected ML model (**Finetuned RoBERTa-base transformer with SMOTE-balanced dataset**) using FastAPI, Streamlit, and Docker. The deployed model can be integrated with social media apps and browser extensions for real-time fake news detection,

6.2 Recommendations

- **Collaborate with Media Outlets** : Work with news organizations and social media platforms to incorporate this model into their systems for real-time fake news detection.
- **Educate the Public** : Raise awareness about fake news detection tools, and encourage people to use these technologies for more informed decision-making.
- **Expand into Multiple Languages** : Ensure the model can detect fake news in various languages to reach a broader audience globally.

6.3 Next Steps

1. Deploy the containerized finetuned RoBERTa model via AWS SageMaker for low-latency inference.
2. Develop browser extensions and social media plugins using the model's API for real-time credibility scoring.
3. Incorporate automation frameworks to flag low-confidence predictions for human review.

